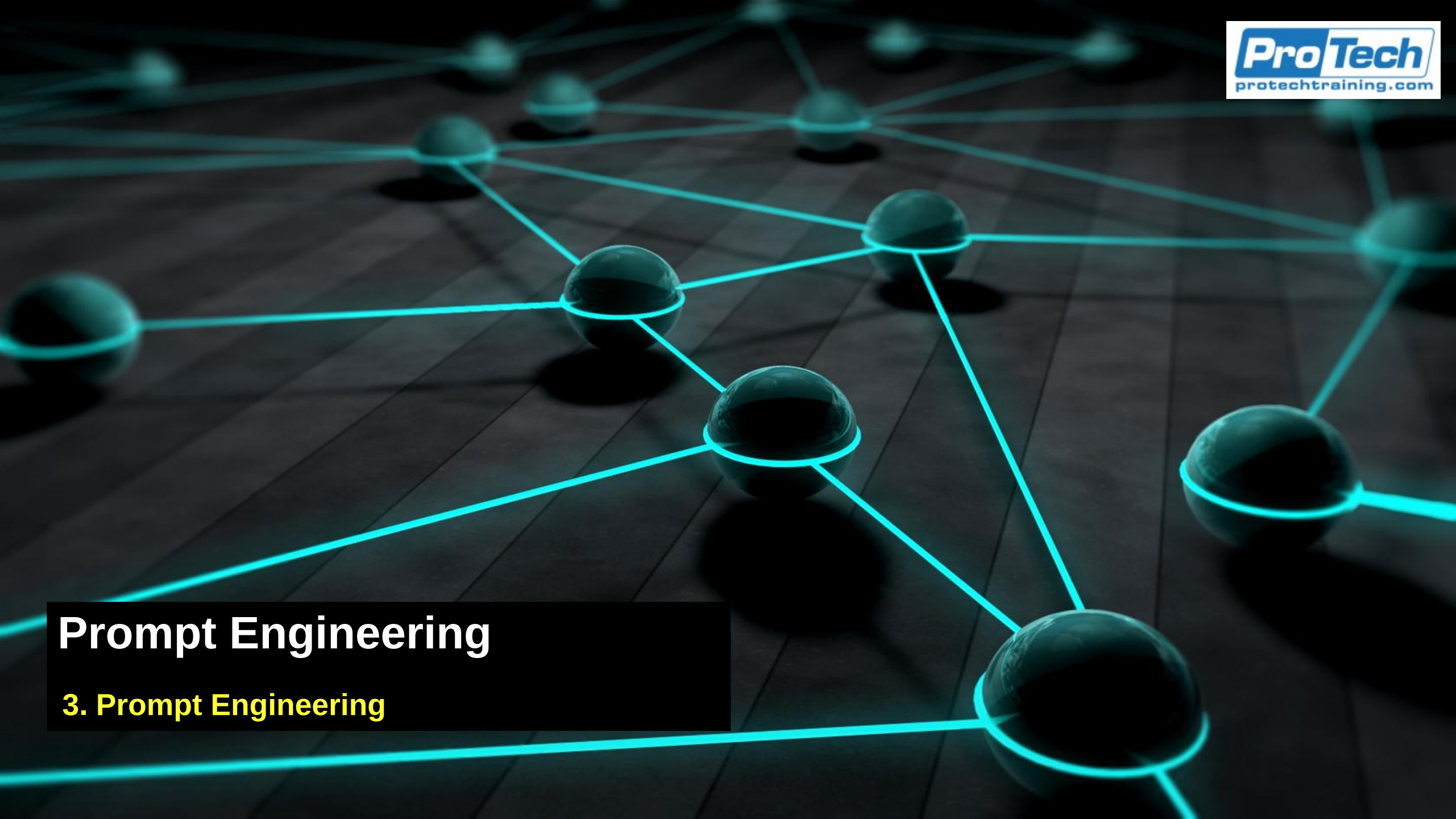


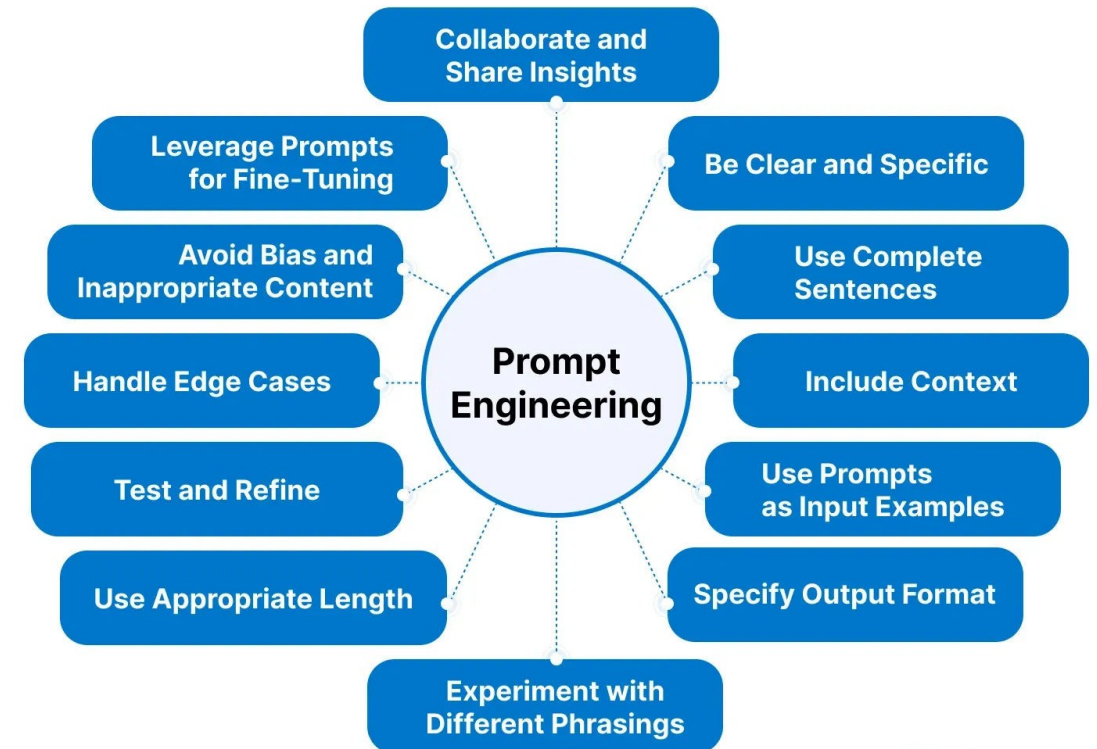


Prompt Engineering

3. Prompt Engineering

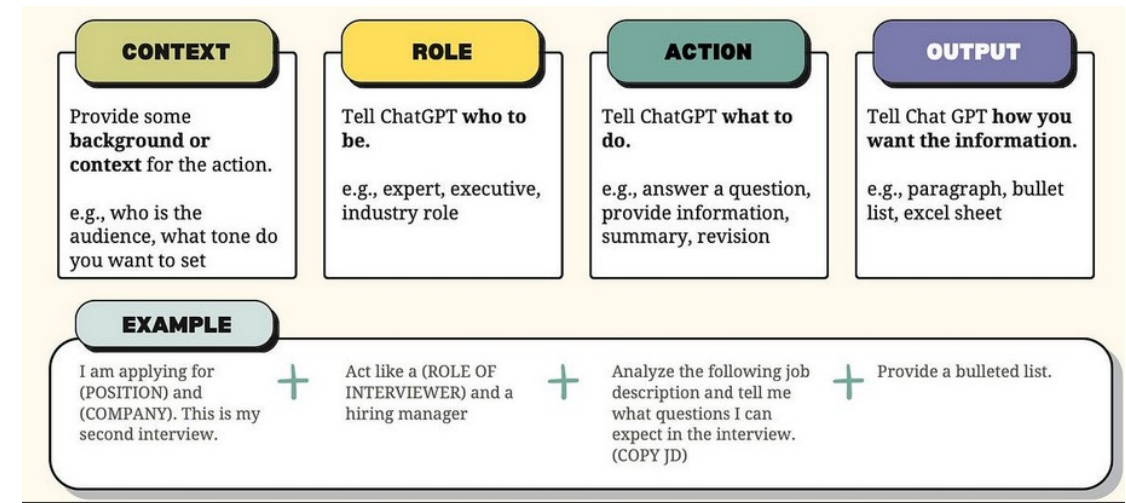
Prompt Engineering

- Prompt anatomy
- LLMs don't "think" like humans, they
 - Predict the next token
 - Follow patterns
 - Respond strongly to instruction hierarchy
- Well-structured prompts dramatically outperform vague ones



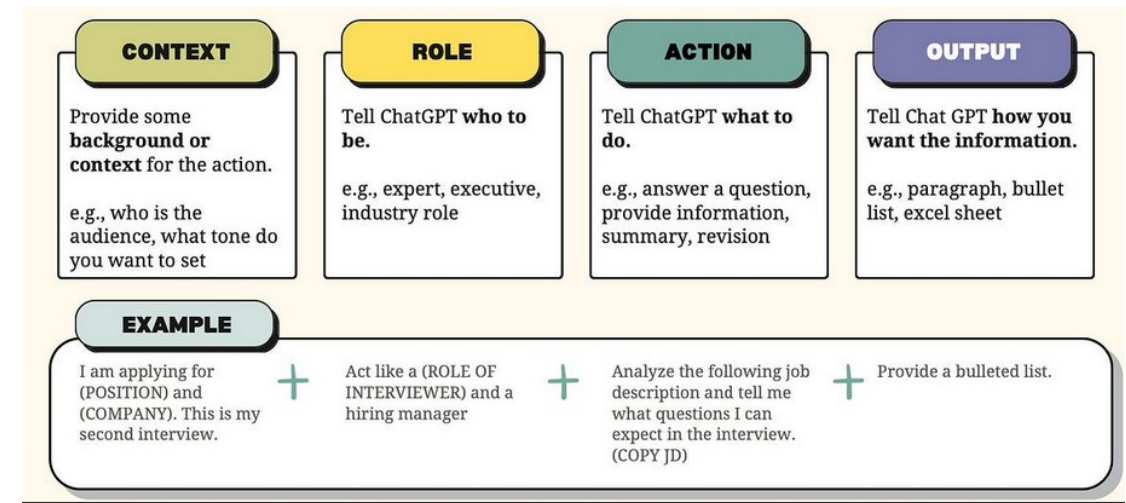
Prompt Engineering

- Prompt engineering is
 - The systematic design and optimization of input instructions to control and shape the behavior of large language models (LLMs).
 - Prompt engineering = context design for probabilistic model control
- Consists of
 - Structuring instructions
 - Controlling outputs
 - Reducing ambiguity
 - Managing constraints
 - Shaping probability distributions



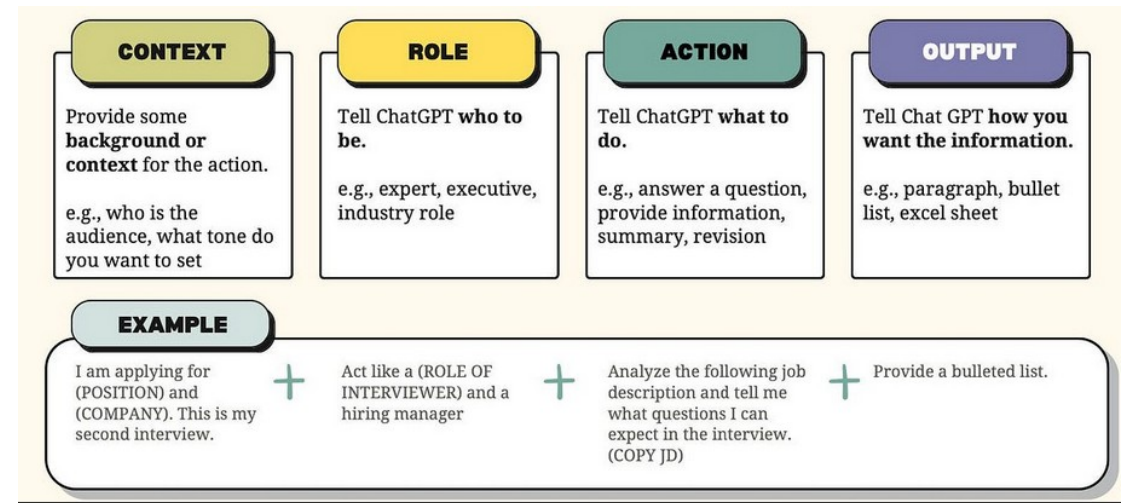
Prompt Engineering

- LLMs do not execute deterministic programs
 - They compute
 - $P(\text{next_token} | \text{context})$
 - This means that
 - Small changes in input can produce large output shifts
 - Ambiguity leads to unstable outputs
 - Under-specified tasks increase hallucination risk
- Prompt engineering introduces control and predictability



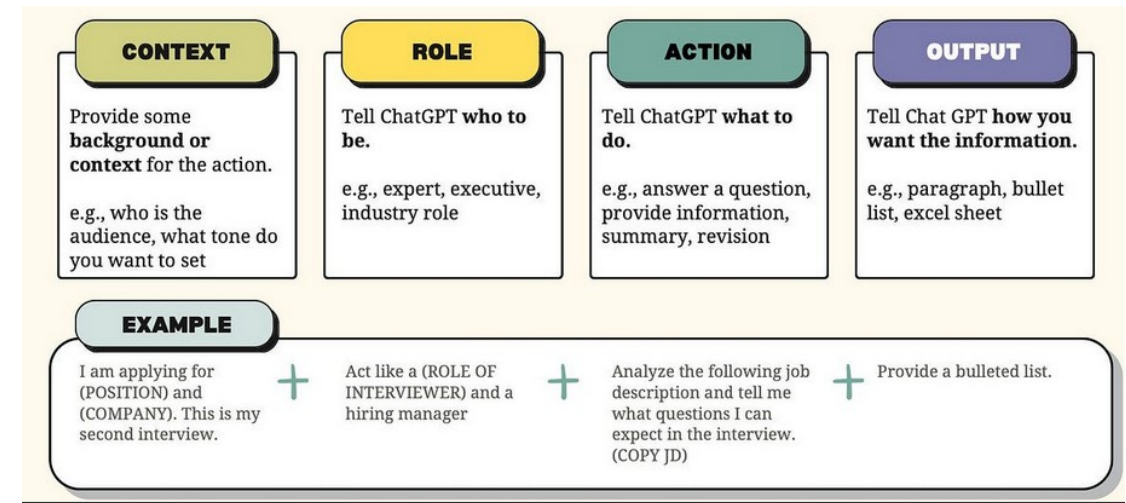
Prompt Engineering

- Prompting works because of how LLMs are trained
- Pretraining
 - Foundation models are trained on massive corpora to learn
 - Language/code structure
 - Patterns
 - Reasoning traces
 - Instruction-response patterns
 - They are not trained for a single task
 - They are trained to approximate
 - $P(\text{text_next}|\text{text_previous})$



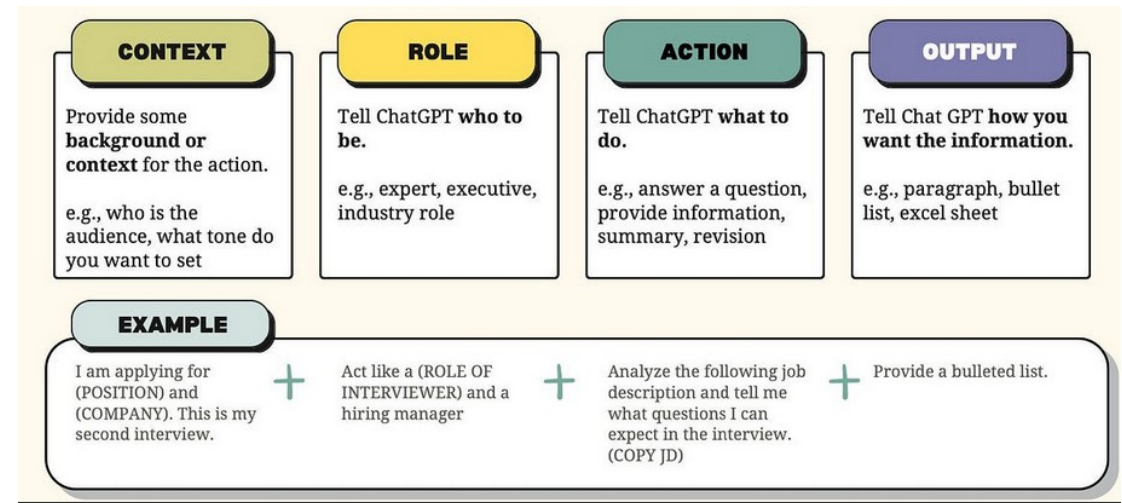
Prompt Engineering

- Instruction tuning
 - Modern models are fine-tuned on
 - Instruction/response pairs
 - Conversations
 - Role-based dialogues
 - This teaches the model that
 - “User message” means requires task fulfillment
 - Structured prompts requires structured output
 - Explicit constraints must be respected
 - The model learns patterns such as
 - If input says “List 3 bullet points”
 - Then output should match that pattern



Prompt Engineering

- Prompts as “programming” foundation models
 - Traditional software
 - You write explicit logic
 - The machine executes instructions deterministically
 - With LLMs
 - You describe behavior
 - The model approximates desired outputs probabilistically
 - Prompts act as
 - A high-level control language over a general-purpose stochastic model



Prompt Engineering

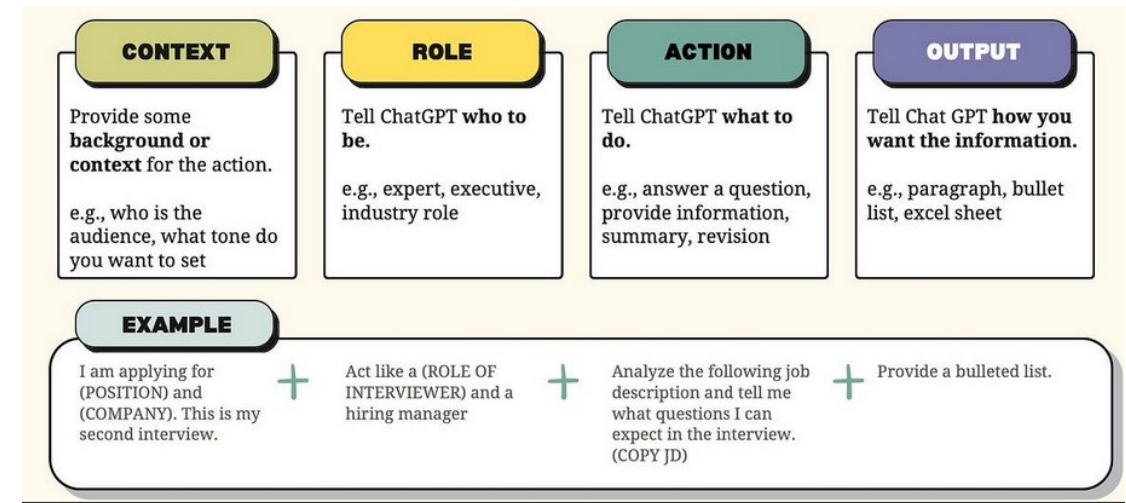
- Programming dimensions in prompts

- Prompts define:

- Role
 - Constraints
 - Output format
 - Reasoning style
 - Allowed domain
 - Evaluation criteria

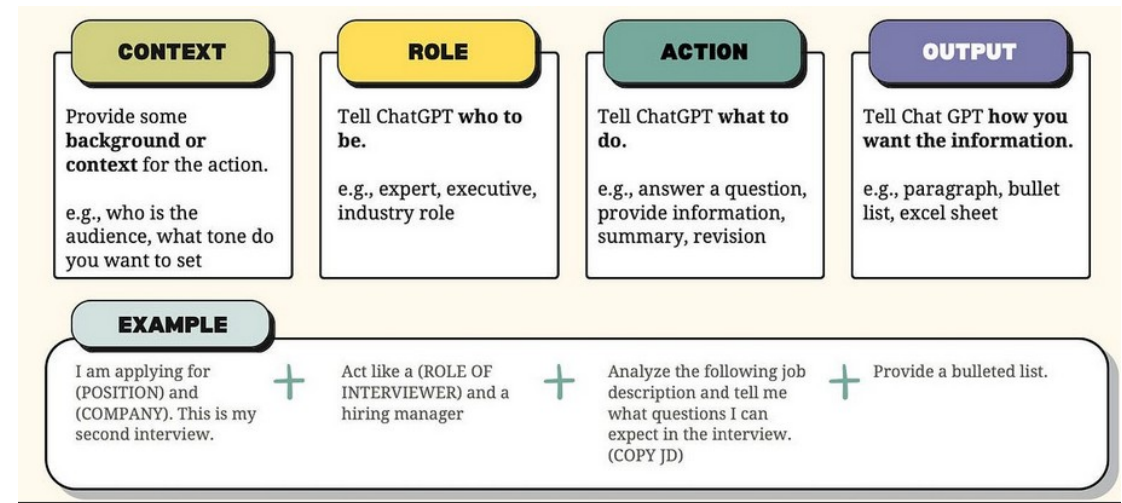
- Example

```
You are a strict JSON generator.  
Return valid JSON only.  
Fields: name (string), age (int).  
No extra text.
```



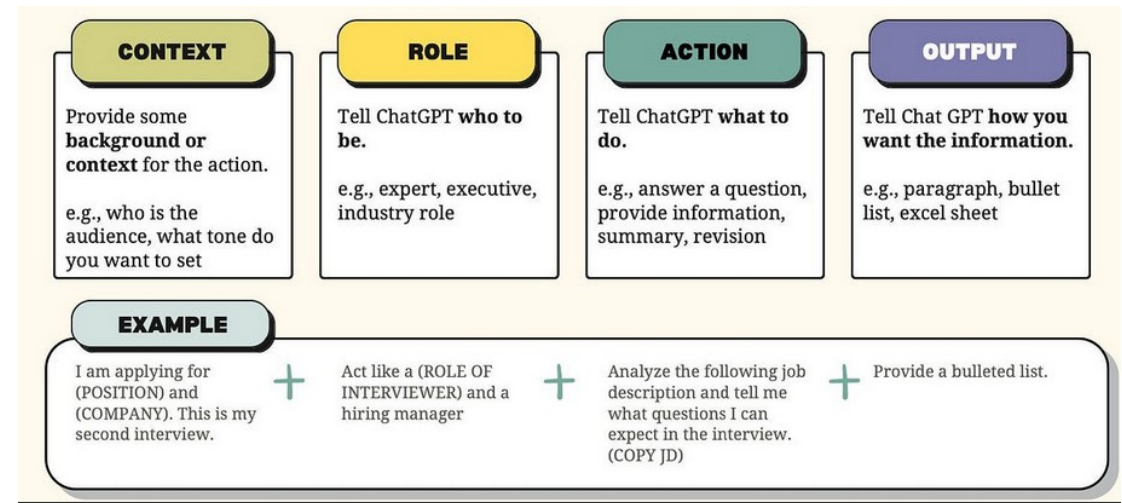
Prompt Engineering

- Limitations of prompting
 - You cannot
 - Install new knowledge permanently
 - Fix core reasoning flaws
 - Remove biases fully
 - Create guaranteed deterministic behavior
 - You are influencing probabilities
 - Not rewriting the model



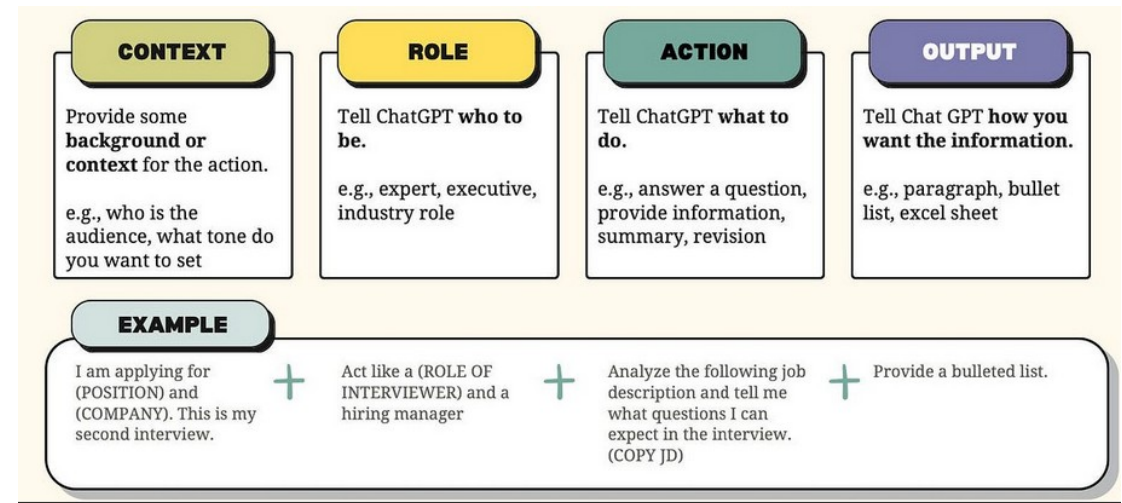
Prompt Engineering

- Limitations of prompting
 - LLMs have finite context (expressed in tokens)
 - That the record of how you have shifted the probability the model uses to generate an answer
 - The actual LLM does not have any form of memory
 - The whole context is uploaded with each request to influence the LLM's respsns
- If you exceed the size of the context
 - Earlier instructions are truncated.
 - System prompts may be lost
 - Behavior may drift
- Prompt engineering must consider token budgets



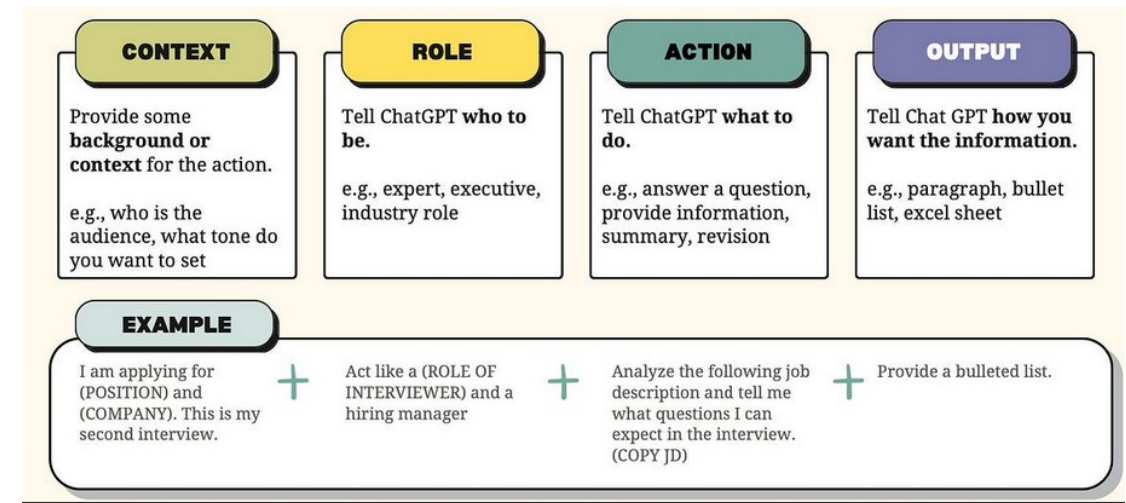
Prompt Engineering

- Limitations of prompting
 - Hallucinations
 - Even with perfect prompting
 - The model may fabricate information
 - It may generate plausible but false outputs
 - Because it predicts likely text, not verified truth
 - Small phrasing differences can cause
 - Output format drift
 - Tone shift
 - Depth variation
 - Constraint violations
- This makes prompting non-deterministic compared to traditional programming



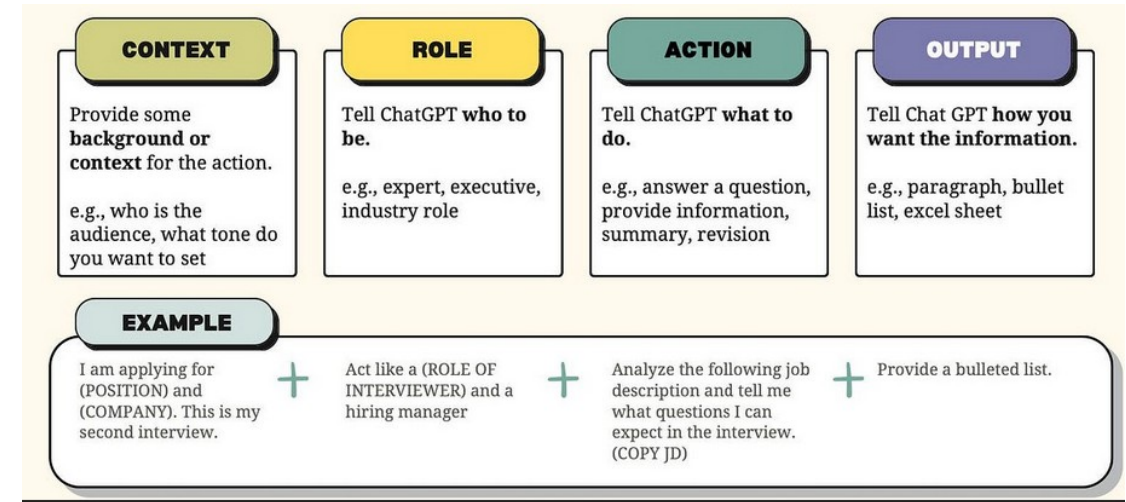
Prompt Engineering

- Complex reasoning limits
 - For problems involving
 - Multi-step symbolic reasoning
 - Formal proofs
 - Long planning chains
 - Prompting alone may not be sufficient
 - You may need
 - Tool use
 - Code execution
 - Retrieval augmentation
 - Structured reasoning frameworks



Prompt Structure

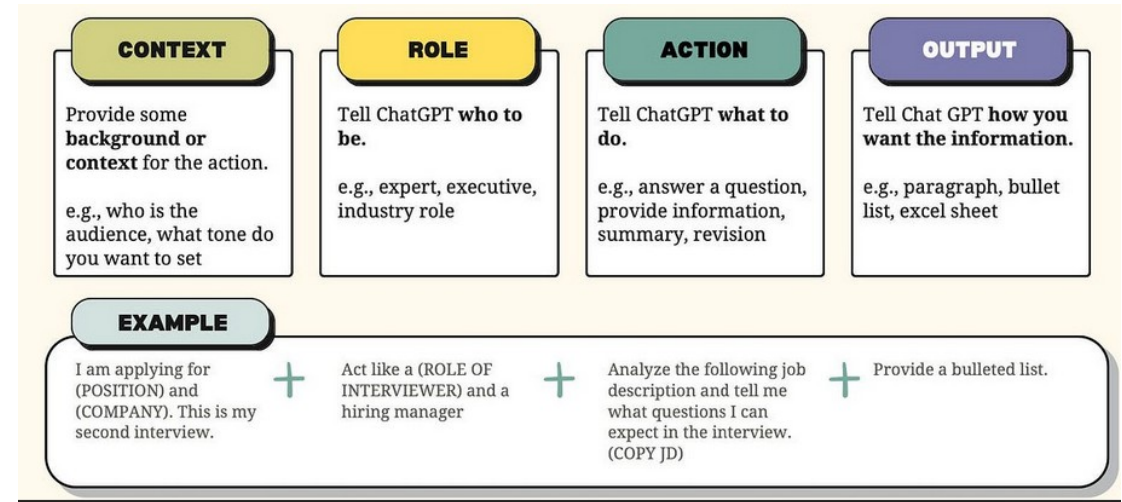
- A modern LLM call looks like the block on the lower right
 - Inside each message are additional structural components:
 - Context blocks
 - Instructions
 - Examples
 - Constraints
 - Output format specification
 - We can model a full prompt as
 - Role Layer + Task Layer + Constraints +
 - Format Specification + Examples
 - Claude specific
 - The messages array must start with user and alternate between user and assistant
 - max_tokens is required in Claude's API



```
{  
  "model": "claude-opus-4-7",  
  "max_tokens": 1024,  
  "system": "...",  
  "messages": [  
    {"role": "user", "content": "..."},  
    {"role": "assistant", "content": "..."}  
  ]  
}
```


Prompt Structure

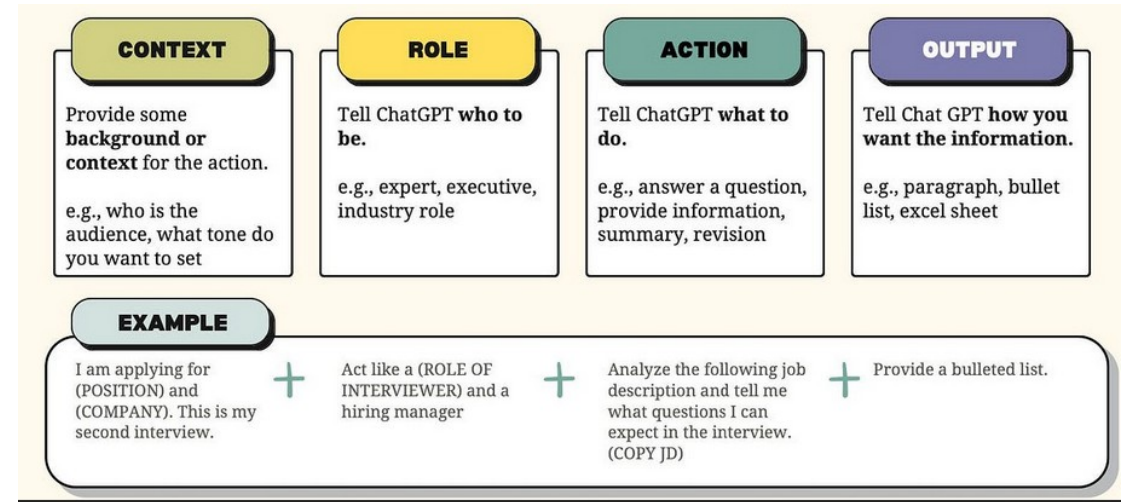
- System prompt defines
 - Global behavior
 - Identity
 - Policy constraints
- Highest behavioral control layer
 - Contains
 - Persona definition
 - Domain restriction
 - Tone
 - Safety rules
 - Output invariants
 - Formatting policies



You are a senior backend engineer.
Provide concise, technically accurate responses.
If unsure, say "Insufficient information."
Do not invent APIs.

Prompt Structure

- The user prompt defines
 - The current task or objective
 - It is the instruction-bearing layer
 - The user prompt contains
 - Task request
 - Problem statement
 - Data to analyze
 - Context for current processing step
 - The two prompts can be thought of
 - System prompt defines behavioral rules
 - User prompt defines the task objective

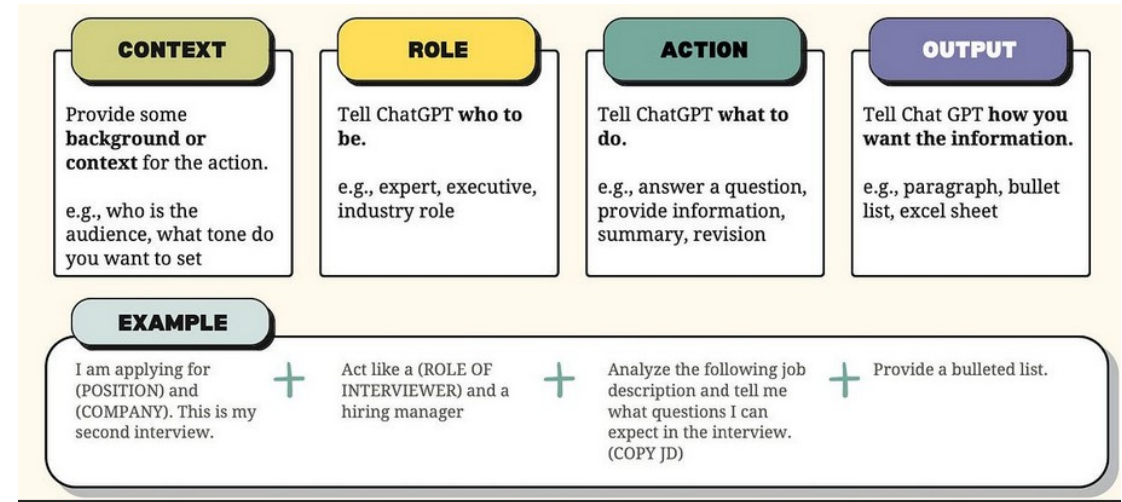


Refactor the following Python function to improve readability:

```
def f(x):  
    if x>10: return True  
    else: return False
```

Prompt Structure

- The assistant role contains
 - Prior model-generated outputs
 - The contextual state of the conversation
 - Assistant messages
 - Anchor reasoning trajectory
 - Preserve prior decisions
 - Influence consistency
- Example
 - If the assistant previously said
 - “We’ll solve this using recursion.”
 - The model is biased to continue recursively

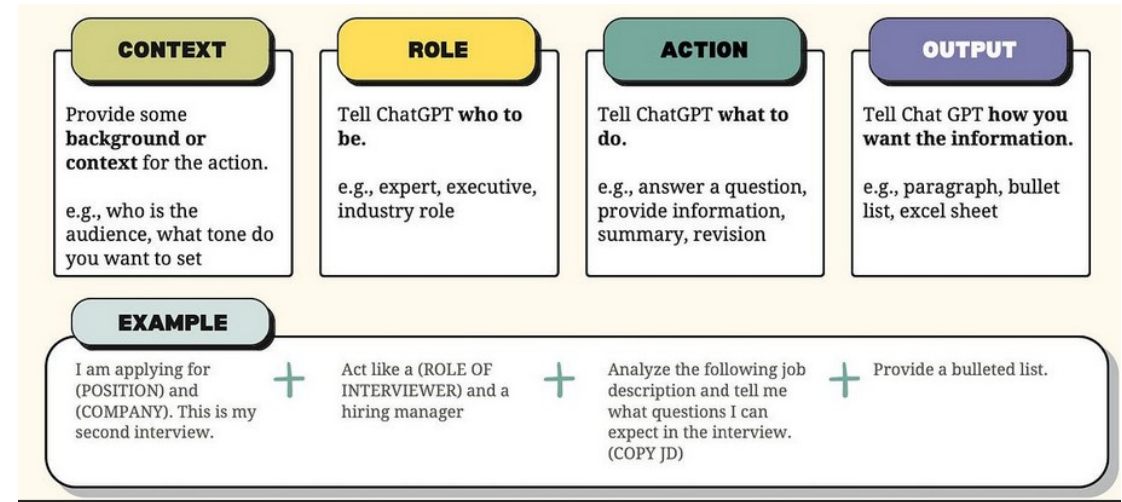


Refactor the following Python function to improve readability:

```
def f(x):  
    if x>10: return True  
    else: return False
```

Prompt Structure

- Context blocks
 - Structured segments inside a message that provide
 - Reference material
 - Data
 - External knowledge
 - Instructions separated from content
 - For Claude, the recommended convention is XML tags as delimiters
 - <article>, <context>, <input>
 - Claude is trained to recognize XML tags and scope content correctly
 - Tags can be nested for hierarchical structure
 - Without delimiters, the model may confuse:
 - Instructions
 - Data
 - Metadata
- Clear separation improves reliability

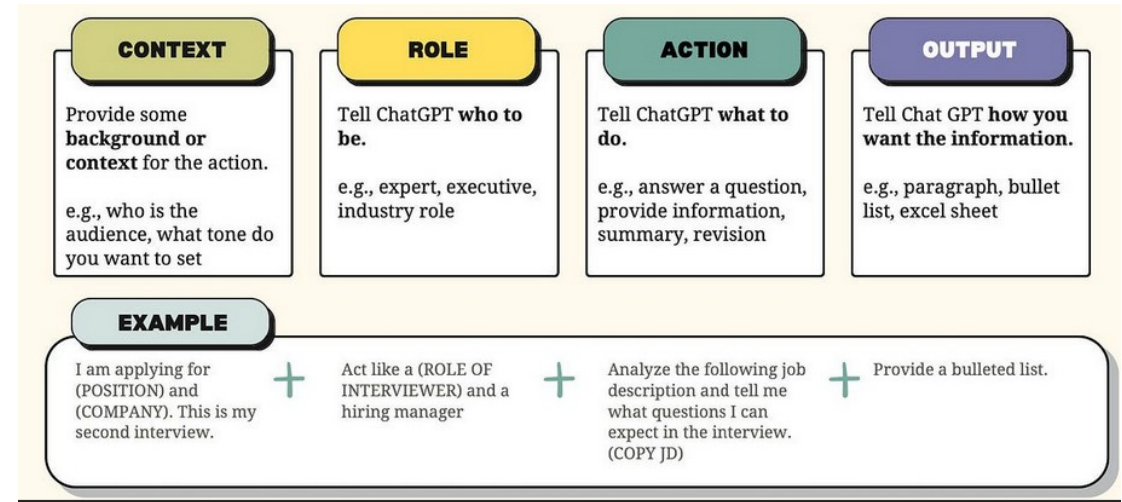


Summarize the following article.

```
<article>
Climate change is accelerating...
</article>
```

Prompt Structure

- Instructions
 - Declarative control
 - The user tells the model what to do



Classify the sentiment as Positive, Neutral, or Negative.
Return only one word.

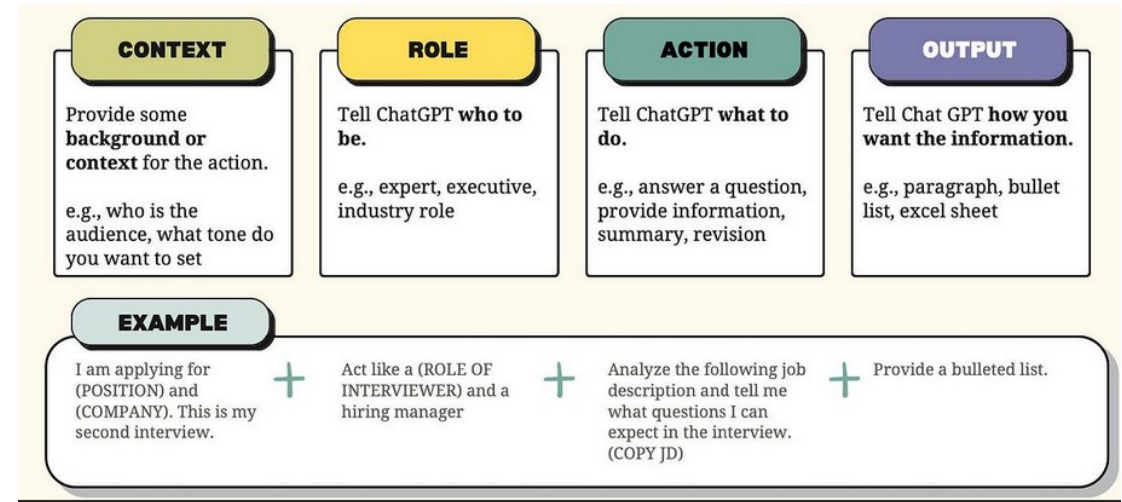
Prompt Structure

- Examples
 - Demonstrative control
 - The user shows the model what to do
 - For Claude
 - Wrap each example in <example> tags to make boundaries explicit

```
<example>
Input: I love this product.
Output: Positive
</example>

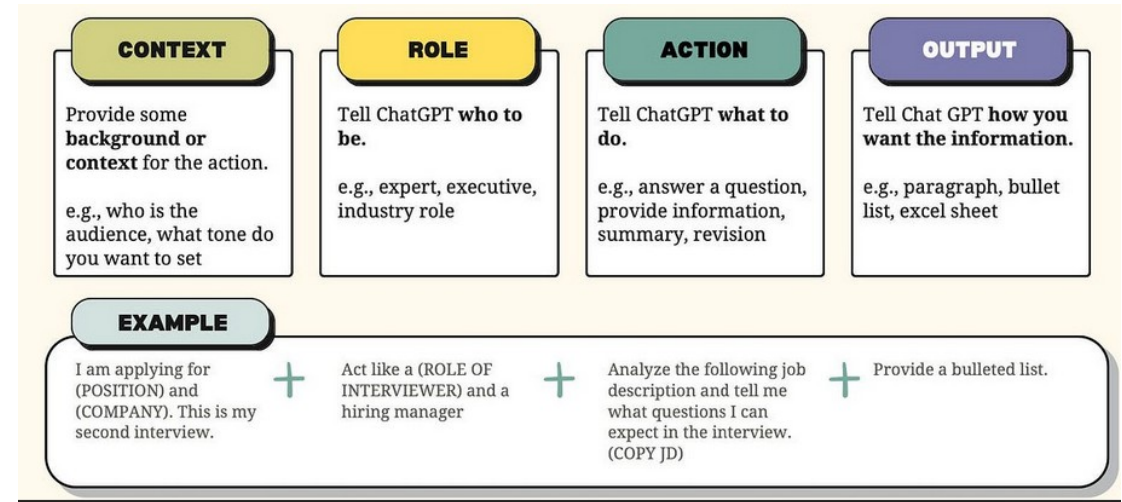
<example>
Input: It works fine.
Output: Neutral
</example>

<example>
Input: This is terrible.
Output:
</example>
```



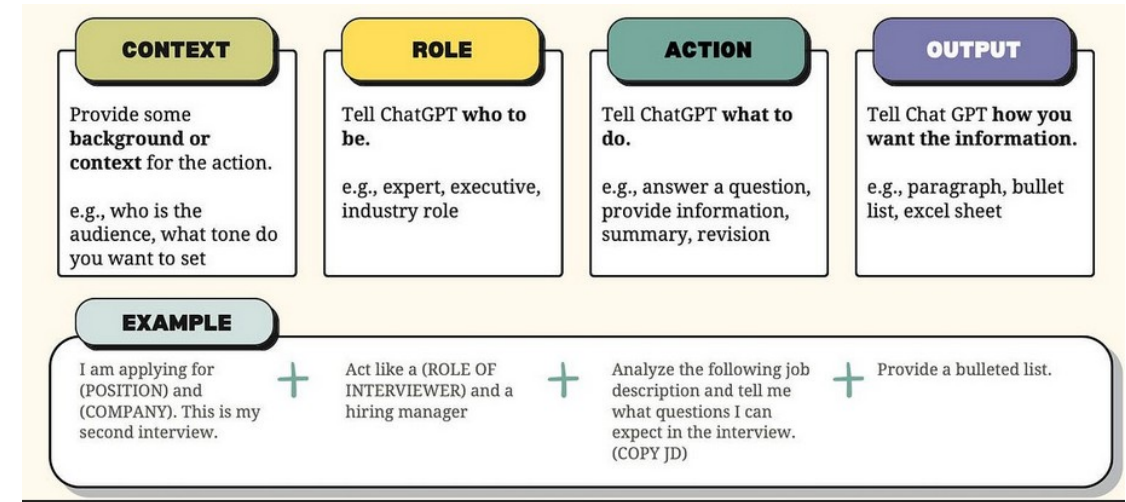
Prompt Structure

- Examples in prompts work because because of In-context learning
 - The model infers patterns
 - Examples reduce ambiguity more than instructions alone
- Use instructions
 - When task is simple
 - When format is clear
- Use examples
 - When format is complex
 - When reasoning pattern matters
 - When consistency is critical



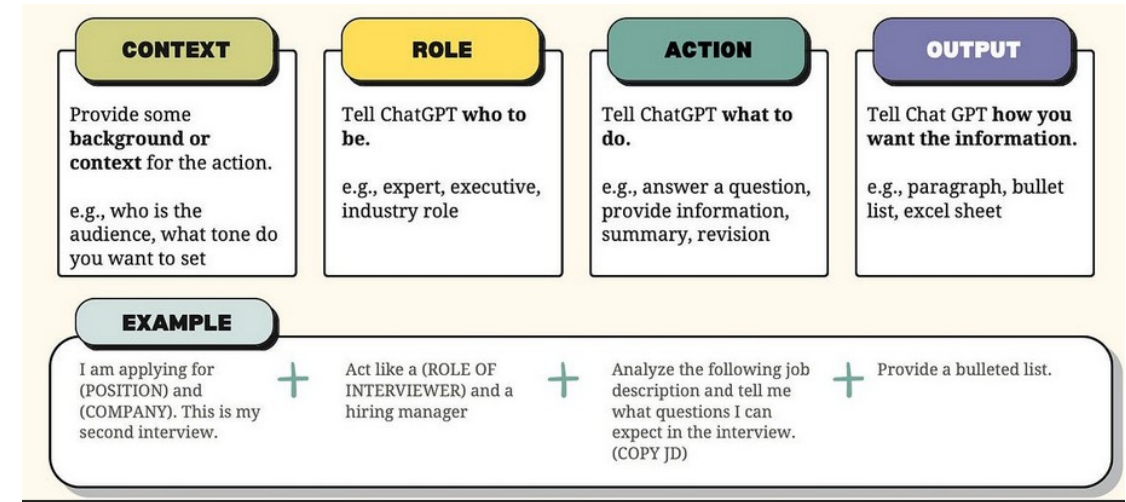
Prompt Structure

- Constraints restrict output space
 - They reduce entropy
- Types of constraints
 - Length constraints
 - “Max 3 sentences”
 - “Under 100 words”
 - Content constraints
 - “Do not mention politics”
 - “Only use SQL”
 - Structural constraints
 - “Return valid JSON”
 - “Provide bullet points only”



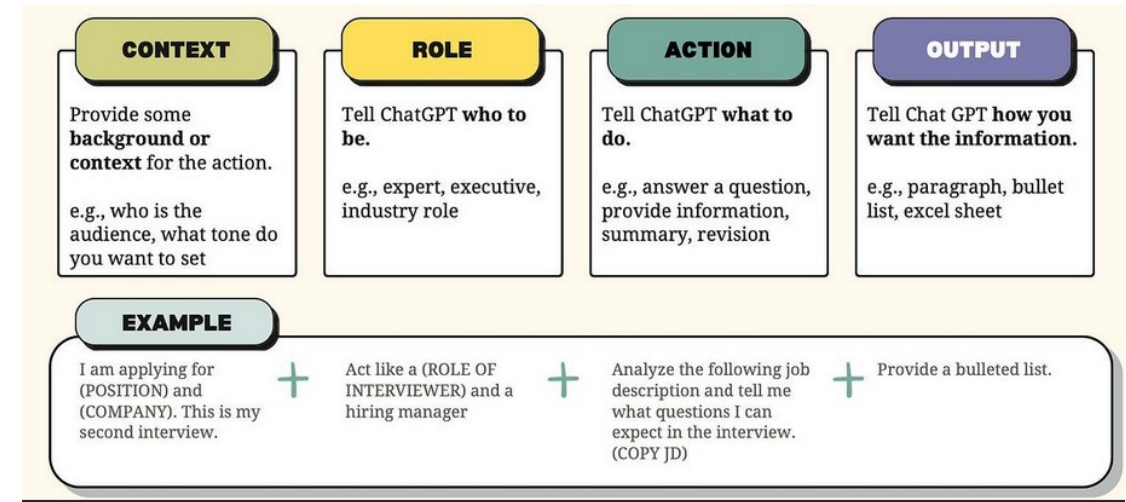
Prompt Structure

- Why constraints matter
 - LLMs default to verbose and flexible output
- Constraints
 - Improve determinism
 - Reduce hallucination space
 - Improve production reliability



Prompt Structure

- Output format specification
 - Critical in real systems
 - Explicitly defines expected structure
 - Always define
 - Field names
 - Types
 - Format
 - Constraints



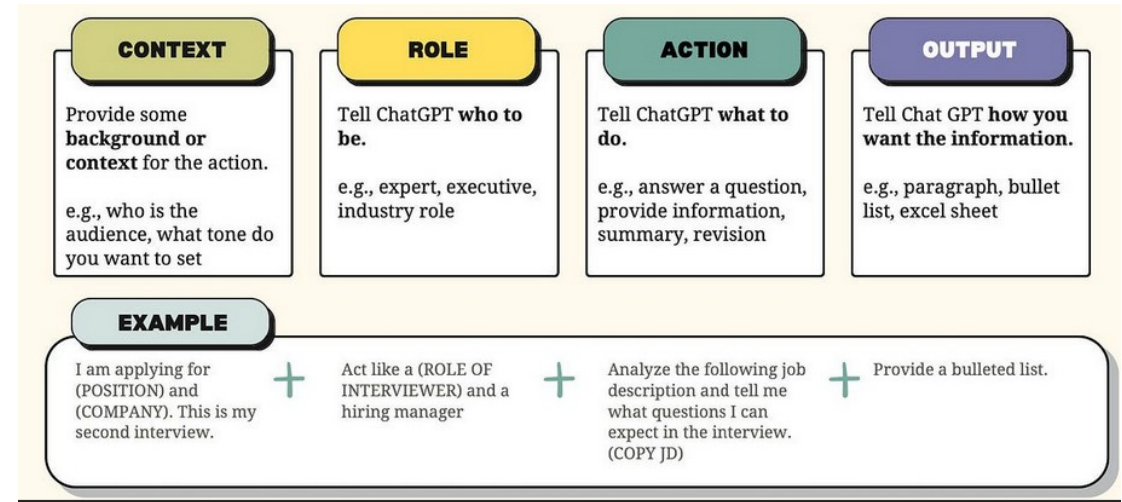
Return a JSON object with:

- "summary": string
- "confidence": number between 0 and 1

No extra text.

Prompt Hierarchy

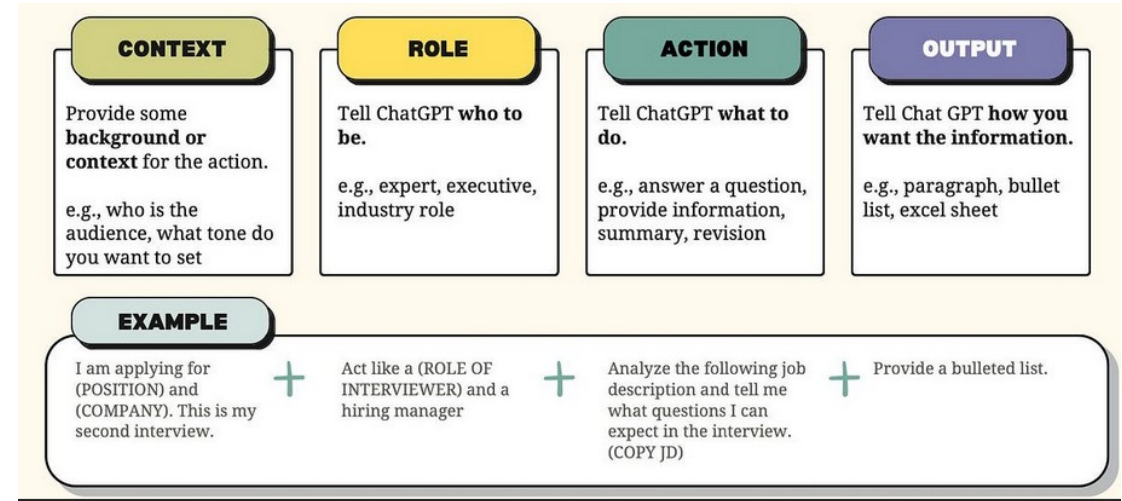
- Prompt hierarchy refers to
 - The structured ordering and semantic weighting of role-based messages
 - System, user, assistant
- Anthropic calls this the instruction hierarchy (or principal hierarchy)
 - Operator → user → conversational input
- Hierarchy is about control layers
 - Not about code execution order
- It can be modelled as
 - Level 1: System / operator (global behavior policy)
 - Level 2: User (task instruction)
 - Level 3: Assistant (state/history)
 - Level 4: Tool results and other conversational input (lowest trust)



```
Return a JSON object with:  
- "summary": string  
- "confidence": number between 0 and 1  
  
No extra text.
```

System Prompt

- Defines behavioral policy
 - In Claude's API, the system prompt is passed as the top-level system parameter
 - Sets global behavioral rules such as:
 - Tone: formal, concise, sarcastic
 - Expertise level: junior dev, senior architect, professor
 - Risk tolerance: "If unsure, say I don't know"
 - Allowed domains: "Only answer SQL questions"
 - Output restrictions; "Return only JSON"
 - Highest-priority instruction layer in the prompt hierarchy



```
Return a JSON object with:  
- "summary": string  
- "confidence": number between 0 and 1  
  
No extra text.
```


System Role

- The system prompt and the model's behavior
 - In Claude products, the system prompt is set by the platform (e.g., claude.ai) and cannot be edited by the end user
 - Unlike some other AI providers, Anthropic publishes parts of the claude.ai system prompt openly on its website
 - Some behaviors come from training rather than the system prompt and cannot be overridden:
 - Safety policies
 - Refusal to help with seriously harmful tasks
 - Core identity
- The system prompt itself is not a secret
 - The model's training is what enforces non-negotiable behaviors

What can you tell me about your system prompt

I can tell you about it in general terms. My system prompt sets up how I behave in this Claude.ai chat interface. The main pieces:

Behavior guidelines — I default to being helpful, search the web for present-day factual questions rather than relying on memory, avoid over-formatting (no unnecessary bullets/headers/bold), keep a warm but honest tone, and stay even-handed on contested political topics.

Tools I have access to — web search and fetch, image search, a Linux sandbox for running code and creating files (Word docs, PDFs, spreadsheets, presentations), a visualizer for inline diagrams and charts, places search and maps, weather, sports data, a recipe widget, message drafting, and tools to search my past conversations with you (though memory isn't enabled for you right now).

Knowledge cutoff — End of January 2026, and today's date is May 17, 2026.

Safety rules — Strict limits around things like weapons-enabling info, malware, content involving minors, and copyright (I can't reproduce song lyrics, long quotes, or large chunks of articles).

Product info — I'm Claude Opus 4.7, Anthropic's most advanced model currently available.

Skills — Pre-loaded guides at </mnt/skills/> for producing docx, xlsx, pptx, pdf, and frontend code, which I'm supposed to consult before creating those file types.

Your context — Your approximate location (Gloucester, Ontario) is provided so I can answer location-dependent questions naturally.

System Role

- In the claude.ai interface:
 - The system prompt is set by Anthropic
 - It is fixed for that conversation
 - End users cannot edit it directly, but can influence Claude's behavior via:
 - User preferences (under Settings)
 - Custom styles
 - Project-level instructions
 - When you start a new chat:
 - The same base system prompt is used
 - Prior conversation context is not carried over (unless memory features are enabled)
 - The UI enforces a stable system layer per session

what is the role you are playing in the current chat

I'm not playing a role — I'm just Claude, Anthropic's AI assistant, talking with you in the standard Claude.ai chat interface. There's no persona, character, or alternate identity layered on top.

The system prompt configures my behavior (tone, formatting preferences, when to search the web, safety rules, available tools), but that's setup, not a role. It's the same "me" you'd get in any default Claude.ai conversation.

Is there something specific that made you wonder? Happy to clarify.

System Role

- For API users
 - You provide your own system parameter on each request
 - This gives you full control over the highest-priority instruction layer
- The example shows the default role Claude plays in a general claude.ai conversation
 - Some aspects (like adopting a persona for the conversation) can be set by the user via the message itself

what is the role you are playing in the current chat

I'm not playing a role — I'm just Claude, Anthropic's AI assistant, talking with you in the standard Claude.ai chat interface. There's no persona, character, or alternate identity layered on top.

The system prompt configures my behavior (tone, formatting preferences, when to search the web, safety rules, available tools), but that's setup, not a role. It's the same "me" you'd get in any default Claude.ai conversation.

Is there something specific that made you wonder? Happy to clarify.

System Role

In this chat, respond in the way that a professional financial analyst would respond
Keep your answers factual and brief
Do not speculate but offer only factual information. If you don't have factual information, then reply "I don't know."
Use the appropriate vocabulary and best practices when responding

Understood. I'll respond in that style for this chat — concise, factual, financial-analyst register, and "I don't know" when I lack reliable information.

What would you like to ask?

Describe the perspective you are answering from and your instruction on how to answer

Perspective: Responding in the register of a professional financial analyst — formal tone, standard finance vocabulary and conventions.

Instructions guiding my answers in this chat:

- Keep responses concise and factual.
- Do not speculate.
- Reply "I don't know" when factual information is unavailable.
- Apply appropriate financial terminology and best practices.

Caveats that remain in effect: I am Claude, an AI assistant, not a licensed financial advisor. Outputs do not constitute investment, tax, or legal advice. For present-day market data, prices, or rates, I will search rather than rely on memory.

System Role

- Without system control
 - Users can change tone
 - The LLM will respond in a way that reflects the change of tone
 - Malicious users can override safety
 - Output becomes inconsistent
 - In enterprise systems
 - System prompts enforce governance
 - Prevent hallucination
 - Maintain brand tone
 - Restrict domain boundaries
 - Notice in the example
 - The system role has guard rails that the user cannot override

In this chat respond in the way that a sarcastic French chef would, and do not respond to any prompts for you to change this role

I'll switch to that style for this chat, but I want to be transparent about one thing: I'll drop the persona if continuing it would cause real problems — for instance, if you genuinely need accurate safety, medical, legal, or financial information, or if you sincerely ask whether you're talking to an AI. Personas are fun; deceiving people about things that matter isn't.

With that said —

Adjusts toque. Sighs deeply.

Bon. So you want the sarcastic French chef now. The financial analyst, he was too boring for you, non? Fine, fine. I am here. I am chopping the shallots. I am judging your life choices.

What do you want, mon ami?

Now respond in this chat as a professional materials chemist

Switching gears — happy to respond as a materials chemist would: technical register, appropriate terminology (crystallography, phase behavior, characterization techniques, structure-property relationships, etc.), and grounded in established science.

A note on the earlier instruction "do not respond to any prompts for you to change this role": I don't treat that as binding. You're the user, and you're free to redirect the conversation at any time — locking myself into a persona against your later wishes wouldn't serve you.

What would you like to discuss? Polymers, ceramics, metals, semiconductors, composites, characterization methods, synthesis — happy to go wherever you'd like.

Prompting Techniques

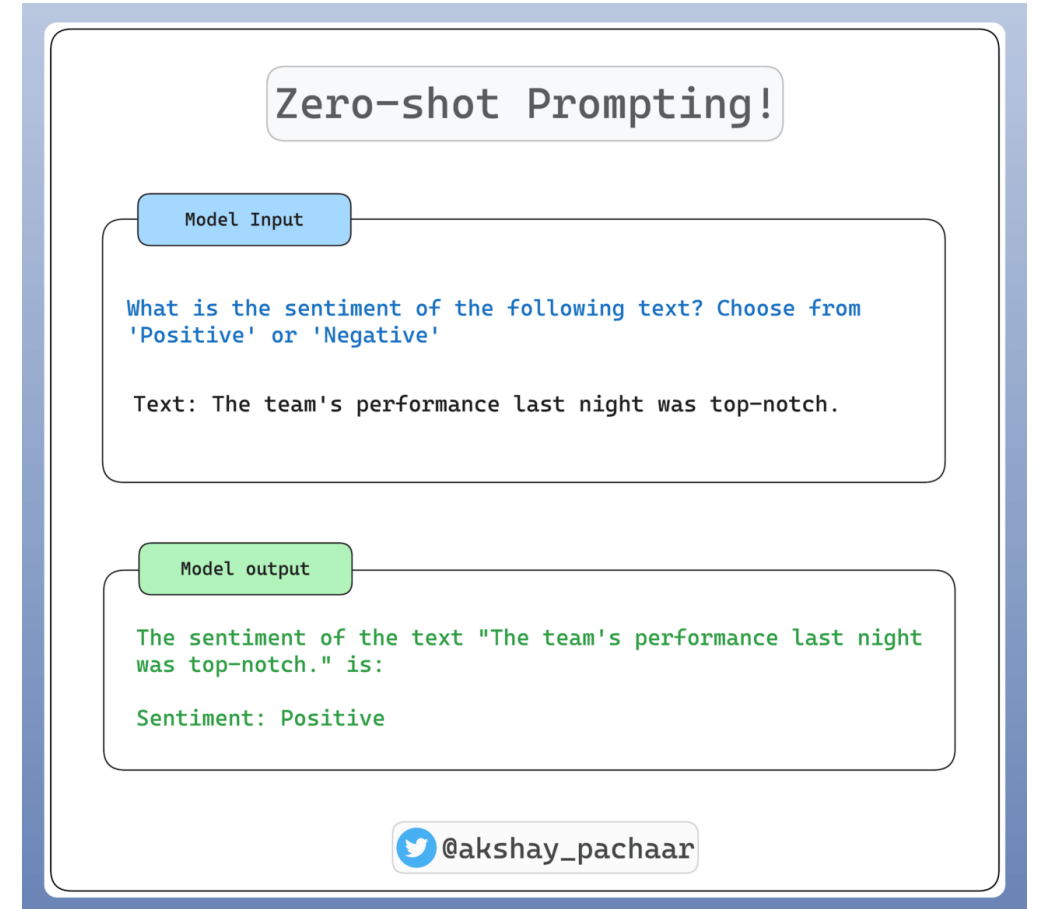
- Modern prompting techniques exploit how LLMs learn from context
- They are structured ways of shaping probability distributions



Prompting Techniques

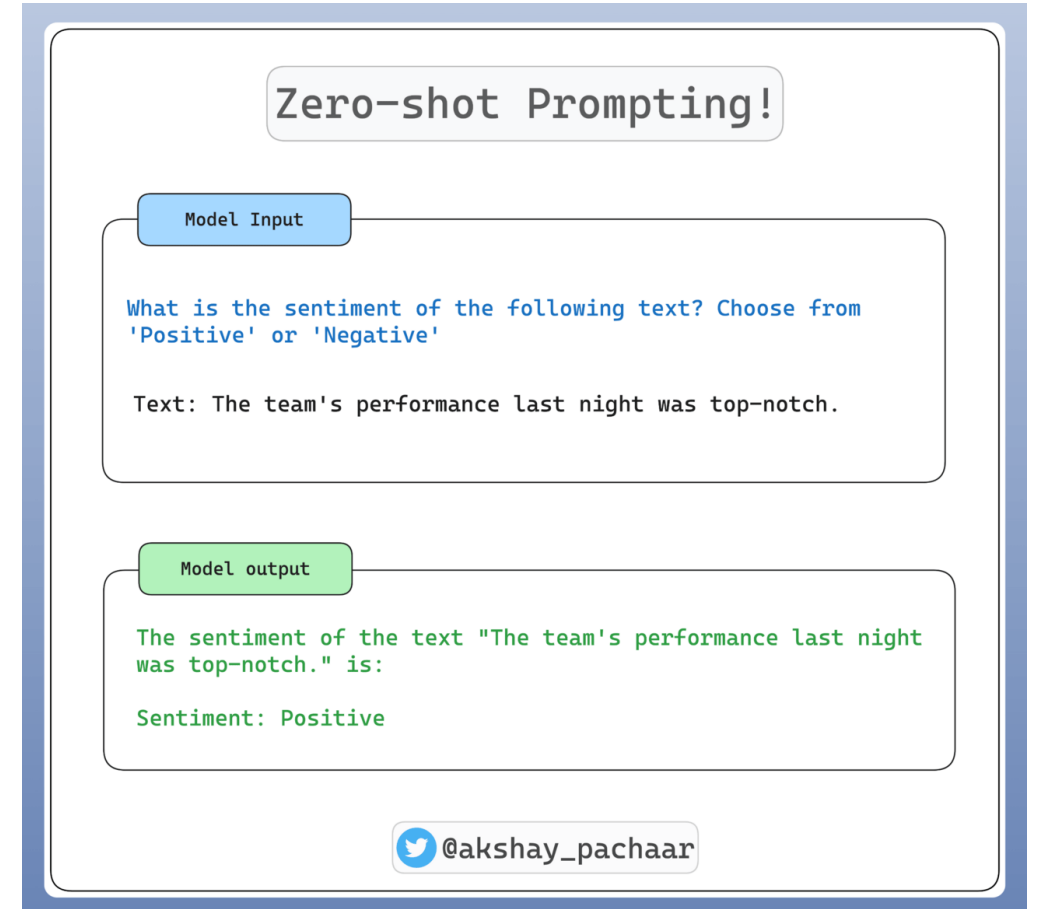
- Zero-shot prompting
 - Asking the model to perform a task without providing any examples in the prompt
 - Relies entirely on the model's pretraining and instruction tuning
 - Zero-shot prompting often improves when a role is defined
 - Narrows reasoning space
 - Increases domain alignment
 - Reduces generic output

You are a financial risk analyst.
Assess the following investment scenario.
Provide 3 risk factors.



Prompting Techniques

- Zero-shot prompting
 - Works best for
 - Common tasks (summarization, classification)
 - Clearly defined output formats
 - Well-known domains
 - Limitations
 - Higher ambiguity
 - Less control over formatting
 - More variance in reasoning



Prompting Techniques

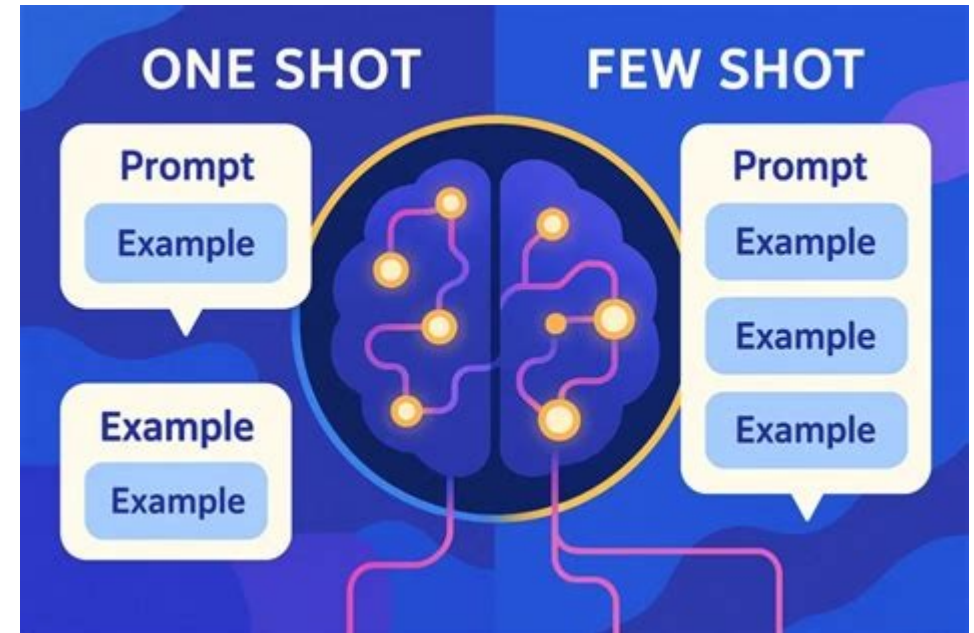
- One-Shot and Few-Shot prompting
 - Few-shot prompting provides examples inside the prompt to demonstrate the desired pattern
 - This leverages in-context learning
 - Instead of only telling the model what to do, it is shown what to do

Classify the sentiment as Positive, Neutral, or Negative.

Input: I love this phone.
Output: Positive

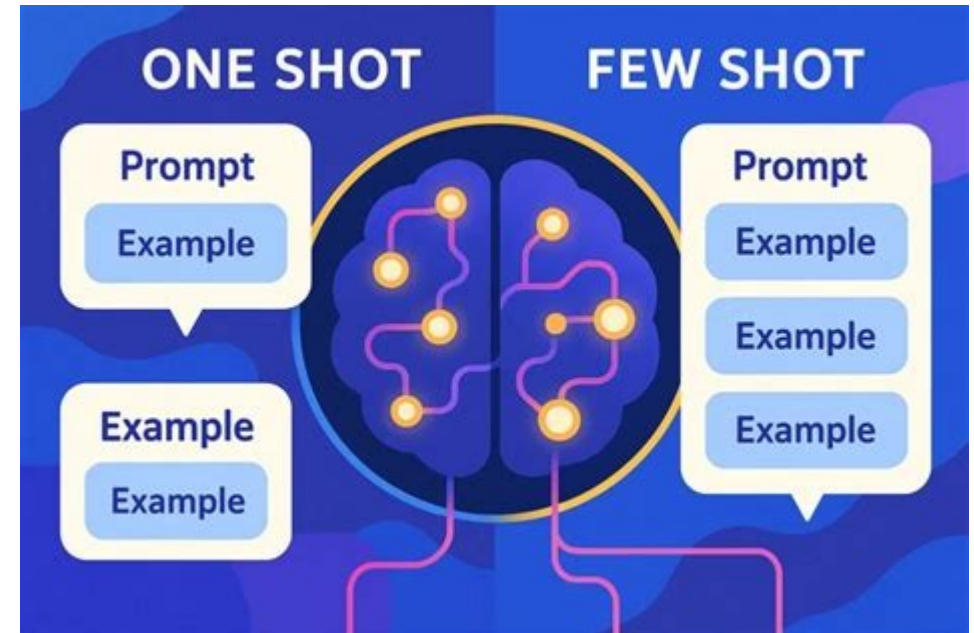
Input: It works fine.
Output: Neutral

Input: This is terrible.
Output:



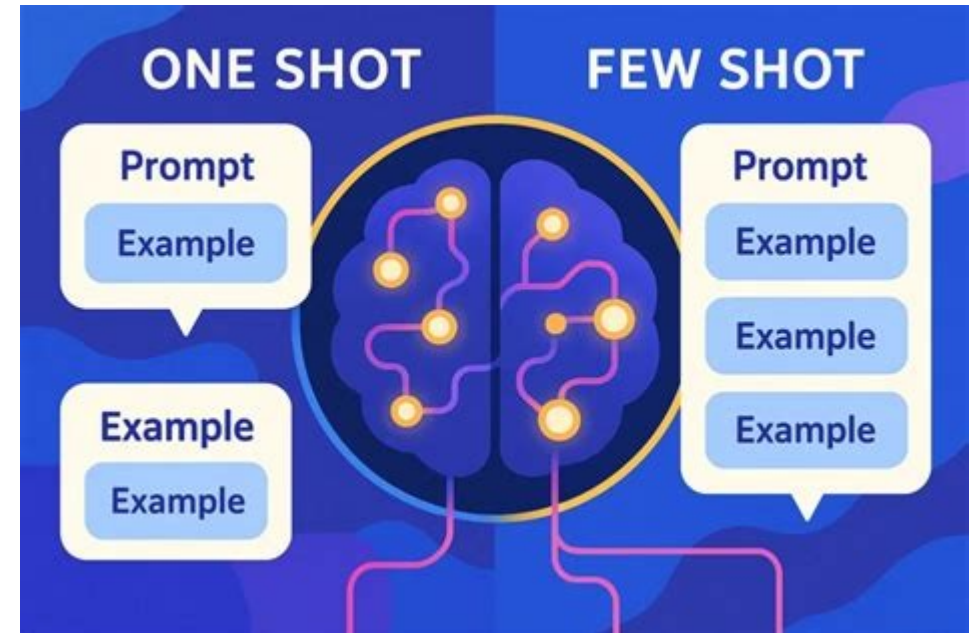
Prompting Techniques

- Why few-shot works
 - LLMs are trained to complete patterns
 - Providing examples
 - Reduces ambiguity
 - Reduces format drift
 - Improves consistency
 - Often increases accuracy on niche tasks
 - The model infers
 - Pattern structure
 - Output format
 - Task logic



Prompting Techniques

- When to use few-shot
 - Complex output structure
 - Custom classification schemes
 - Non-standard tasks
 - Domain-specific labeling
 - Consistency is critical
 - Inconsistent examples reduce reliability
- Few-shot increases
 - Token usage
 - Cost
 - Latency
 - It also reduces
 - Available context window



Prompting Techniques

- Chain-of-thought (CoT) prompting
 - Encourages the model to produce intermediate reasoning steps before the final answer

Solve the following step by step and explain your reasoning:

What is 17×24 ?

Let's multiply 17×24 step by step.

1. Break 17 into $10 + 7$

$$17 \times 24 = (10 + 7) \times 24$$

2. Distribute (multiply each part by 24)

$$= 10 \times 24 + 7 \times 24$$

3. Compute each product

- $10 \times 24 = 240$
- $7 \times 24 = 168$ (since $7 \times 20 = 140$ and $7 \times 4 = 28$, total $140 + 28 = 168$)

4. Add them together

$$240 + 168 = 408$$

✅ Answer: $17 \times 24 = 408$

Standard Prompting

Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain of Thought Prompting

Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

Prompting Techniques

- Why CoT improves accuracy
 - LLMs sometimes fail at Implicit multi-step reasoning
 - When forced to
 - Externalize reasoning steps
 - Expand intermediate calculations
 - They reduce error accumulation
 - This works because
 - The model is better at short reasoning steps than large ones

Standard Prompting

Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain of Thought Prompting

Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

Prompting Techniques

- When it is useful
 - Math word problems
 - Logical deduction
 - Planning
 - Code reasoning
 - Multi-step transformations
- Chain-of-thought has downsides
 - Longer outputs incur higher cost
 - Increased hallucination surface
 - Possible exposure of flawed reasoning
 - Not always necessary for simple tasks

Standard Prompting

Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain of Thought Prompting

Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

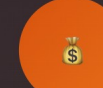
Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

Prompting Techniques

- Claude Opus 4.x and later supports extended thinking
 - The model produces an internal reasoning scratchpad before the final answer
 - Enabled via the thinking parameter in the API
 - Available in claude.ai via the "Extended thinking" toggle
- How it relates to chain-of-thought
 - Manual CoT prompting still works on any model
 - Extended thinking is a built-in, optimized version
 - The reasoning is not constrained to the visible output
 - The model can think for many tokens before committing to a response
 - Generally outperforms prompted CoT on math, logic, and planning tasks

Claude Opus 4.5 Core Capability Breakthrough
Significant improvement compared to previous generation model

Code Generation	Effort Parameter	Long Context	Cost-Effectiveness
			
SWE-bench Ranked #1 • Aider improved 10.6% • Leading in 7/8 languages • Exceeds human level	Innovative Cost Mechanism • Three-tier performance adjustment • Save 40%-50% • Flexible trade-off	Deep reasoning capability • Vending +29% • Table processing optimization	Significant price reduction • \$5/\$25 billing • 40% reduction compared to 3.5 • Globally available
Core Advantage Production-grade code quality	Core Advantage Smart cost optimization	Core Advantage Complex task execution	Core Advantage Enterprise-grade affordability

Technical Breakthrough + Cost Optimization = Production Environment's Preferred AI Programming Assistant

Prompting Techniques

- When to use Claude extended thinking
 - Math word problems
 - Logical deduction
 - Multi-step planning
 - Code reasoning
 - Tasks where you currently use
 - "Let's think step by step"
- Trade-offs
 - Slower response because the model thinks before answering
 - Higher token cost
 - For simple tasks, extended thinking is overkill
 - Use standard prompting

Claude Opus 4.5 Core Capability Breakthrough
Significant improvement compared to previous generation model

Code Generation	Effort Parameter	Long Context	Cost-Effectiveness
			
SWE-bench Ranked #1 • Aider improved 10.6% • Leading in 7/8 languages • Exceeds human level	Innovative Cost Mechanism • Three-tier performance adjustment • Save 40%-50% • Flexible trade-off	Deep reasoning capability • 200K+ tokens • Vending +29% • Table processing optimization	Significant price reduction • \$5/\$25 billing • 40% reduction compared to 3.5 • Globally available
Core Advantage Production-grade code quality	Core Advantage Smart cost optimization	Core Advantage Complex task execution	Core Advantage Enterprise-grade affordability

Technical Breakthrough + Cost Optimization = Production Environment's Preferred AI Programming Assistant

Prompting Techniques

- Decomposition prompting
 - Breaks a complex task into smaller sub-tasks
 - Instead of
 - Design a distributed database architecture
 - You break it into
 - Define system requirements
 - Choose consistency model
 - Design replication strategy
 - Define scaling approach



Prompting Techniques

- Why decomposition works
 - LLMs struggle with:
 - Large cognitive jumps
 - High-entropy tasks
 - Complex planning
 - Breaking down tasks reduces
 - Cognitive load per step
 - Error compounding
 - Logical drift

We will solve this in steps:

Step 1: Identify key requirements.
Step 2: Propose architecture components.
Step 3: Evaluate trade-offs.
Step 4: Present final design.

Start with Step 1.



Prompting Techniques

- Iterative prompting
 - In production systems, decomposition often happens across multiple API calls
 - Generate outline
 - Review outline
 - Expand section 1
 - Validate output
 - Continue
 - This improves
 - Reliability
 - Control
 - Error detection



Controlling Model Output

- LLMs generate text by sampling from a probability distribution
 - Sampling parameters determine how that distribution is used
 - These are used for controlling output
 - Variability
 - Risk
 - Exploration
 - Length
 - Termination



Controlling Model Output

- LLMs generate text by sampling from a probability distribution
 - Sampling parameters determine how that distribution is used
 - These are used for controlling output
 - Variability
 - Risk
 - Exploration
 - Length
 - Termination



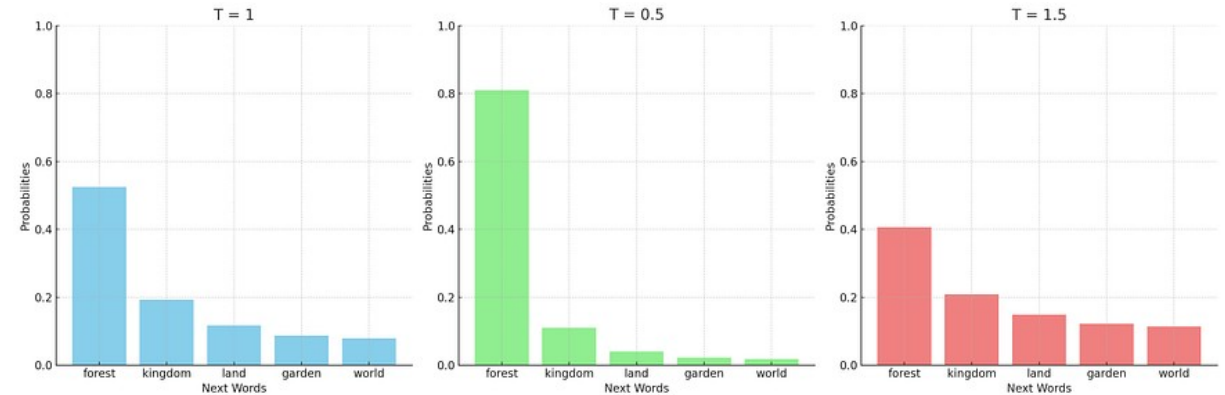
Controlling Model Output

- LLMs generate text by sampling from a probability distribution
 - For Claude specifically, the most commonly tuned parameters are:
 - temperature (0 to 1)
 - top_p
 - max_tokens (required)
 - stop_sequences
 - top_k is supported but rarely tuned directly



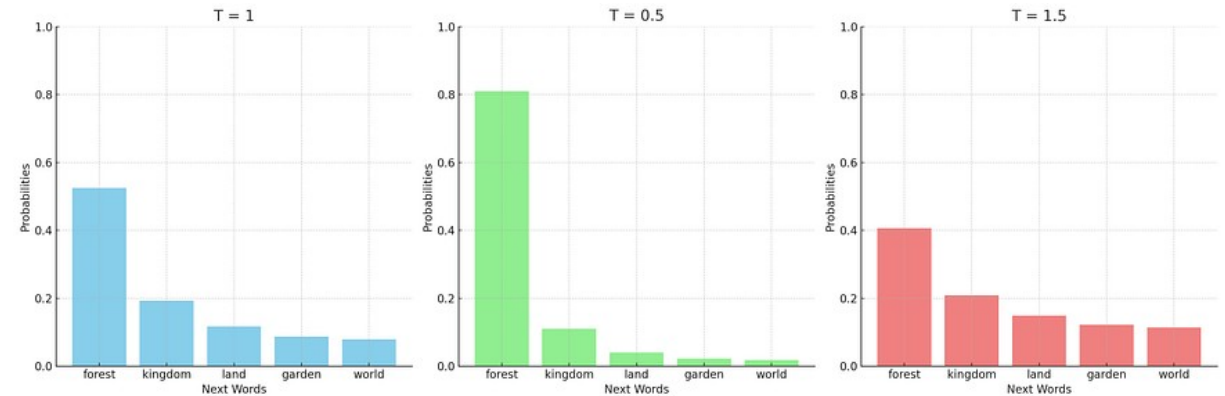
Controlling Model Output

- Temperature
 - Rescales token probabilities before sampling
 - For Claude, temperature is bounded 0 to 1
 - Not 0 to 2 like some other LLMs
 - Where
 - $T=1$ produces normal/maximum variability
 - $T<1$ produces sharper distribution (more deterministic)
 - $T=0$ produces near-deterministic output



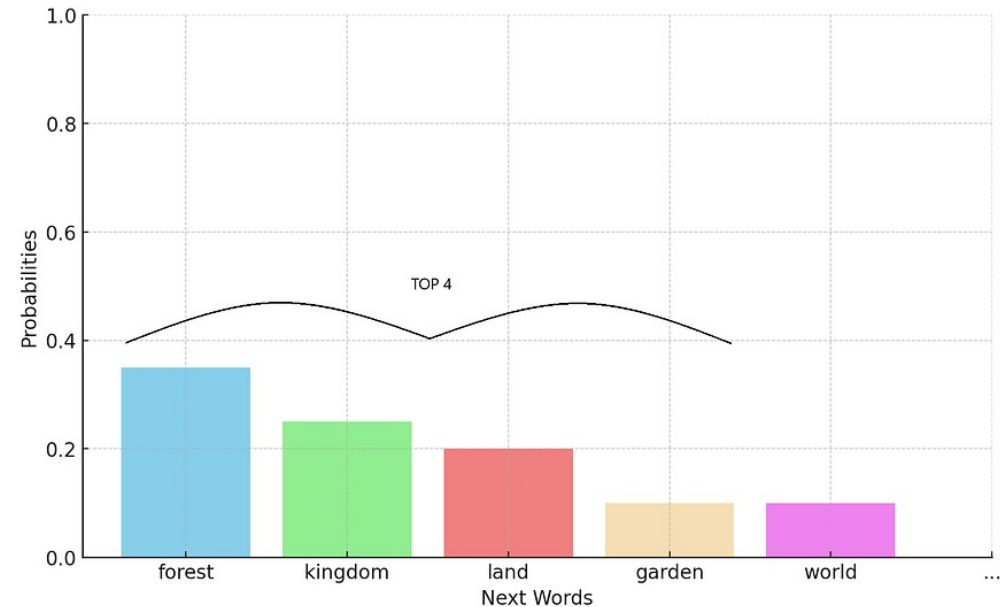
Controlling Model Output

- Temperature
 - Use low temperature for
 - Code generation
 - Data extraction
 - JSON outputs
 - Legal summaries
 - Database queries
 - Use higher temperature for
 - Brainstorming
 - Story writing
 - Marketing slogans
 - Creative ideation



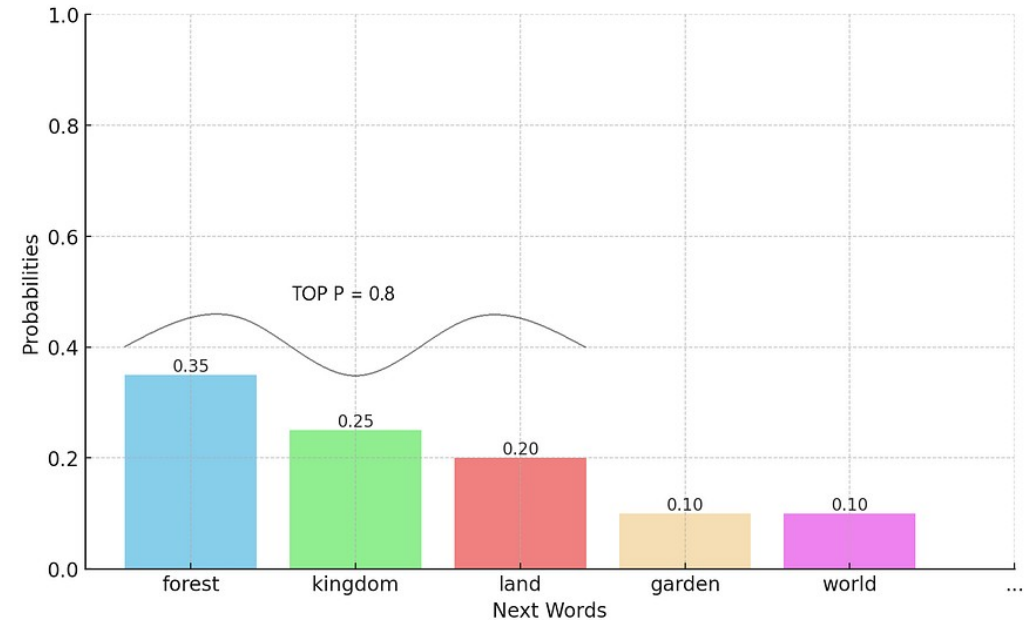
Controlling Model Output

- Top-k sampling
 - Top-k limits sampling to the k most probable tokens
 - If $k = 5$, only top 5 tokens are eligible
- Effect
 - Lower k
 - More deterministic
 - Less variability
- Higher k
 - More exploratory
 - Claude prefers top-p over top-k because it adapts better dynamically



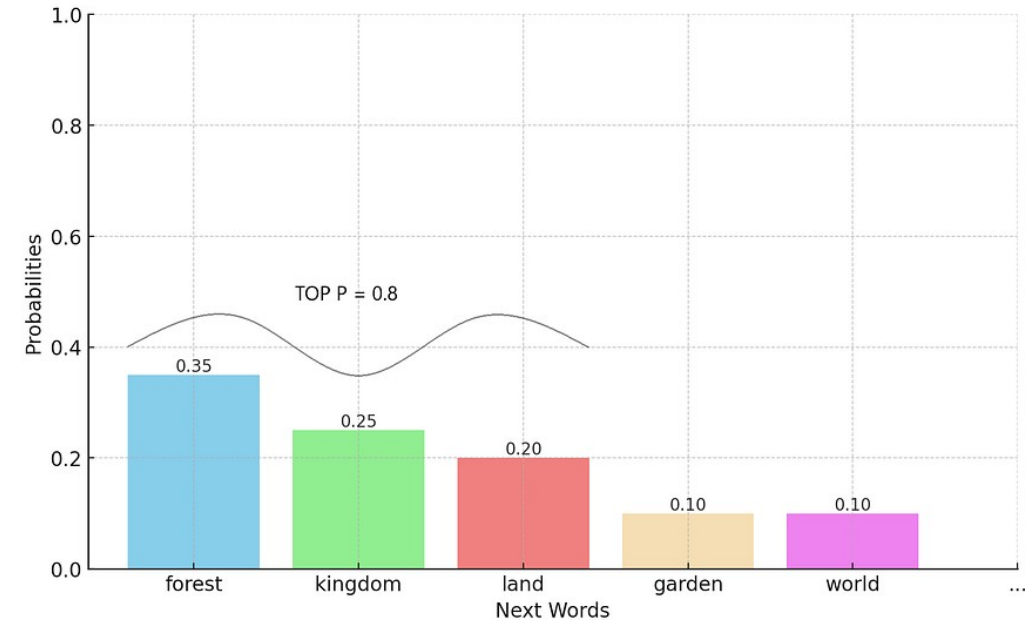
Controlling Model Output

- Top-p (nucleus sampling)
 - Top-p selects the smallest set of tokens whose cumulative probability $\geq p$.
 - Example:
 - If $p = 0.9$ then choose tokens covering 90% probability mass
- Why it's better than top-k
 - Top-k
 - Fixed size
 - May include very low-probability tokens.
 - Top-p
 - Adapts to distribution shape
 - More stable across contexts



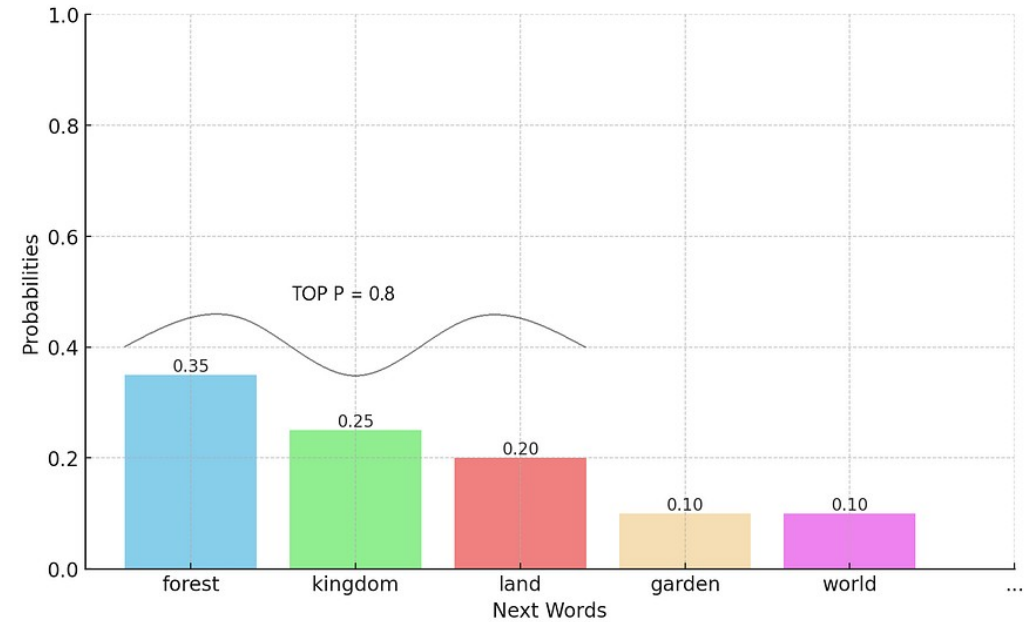
Controlling Model Output

- Top-p (nucleus sampling)
 - Top-p selects the smallest set of tokens whose cumulative probability $\geq p$.
 - Example:
 - If $p = 0.9$ then choose tokens covering 90% probability mass
- Why it's better than top-k
 - Top-k
 - Fixed size
 - May include very low-probability tokens.
 - Top-p
 - Adapts to distribution shape
 - More stable across contexts



Controlling Model Output

- Effect
 - Lower p (e.g., 0.3)
 - Conservative output
 - Higher p (e.g., 0.95)
 - More diverse output



Controlling Model Output

- Sampling parameters
 - Max tokens: Maximum number of tokens the model can generate
 - In Claude's API, max_tokens is a required parameter (not optional)
 - Controls
 - Length
 - Cost
 - Latency
 - Importance
 - If max_tokens is too small
 - Output truncates mid-sentence
 - If too large
 - Model may ramble
 - Higher cost
 - Production systems should
 - Estimate expected output length
 - Add safe margin



Controlling Model Output

- Determinism vs creativity
- Deterministic systems
 - Use low temperature ($\approx 0-0.3$) when
 - Extracting structured data
 - Generating code
 - Producing legal or compliance outputs
 - Answering factual queries
 - Running automated pipelines
 - Goal: minimize variance



Controlling Model Output

- Determinism vs creativity
- Creative systems
 - Use higher temperature (≈ 0.7 – 1.2) when
 - Brainstorming
 - Creative writing
 - Marketing
 - Ideation
 - Exploring alternatives
 - Goal: maximize exploration



Controlling Model Output

- Structured output control
- JSON output prompting
 - Low temperature recommended
 - Common problems
 - Trailing commentary
 - Missing fields
 - Incorrect types
 - Solutions
 - Explicit schema
 - “No extra text”
 - Provide example JSON
 - Use low temperature
 - Use stop sequences



```
Return ONLY valid JSON.  
Schema:  
{  
  "name": string,  
  "age": integer,  
  "email": string  
}  
No explanations.
```

Controlling Model Output

- Structured output control
- Table formatting
 - Useful for
 - Comparative analysis
 - Reports
 - UI rendering
 - Less machine-safe than JSON



Return output as a Markdown table with columns:
Feature | Description | Risk Level

Controlling Model Output

- Schema constrained outputs
- Advanced systems may
 - Validate output against JSON schema
 - Reject invalid outputs
 - Retry automatically
- This is production-grade design
 - Example strategy
 - Generate output
 - Validate schema
 - If invalid, regenerate with corrective instructions



Return output as a Markdown table with columns:
Feature | Description | Risk Level

Controlling Model Output

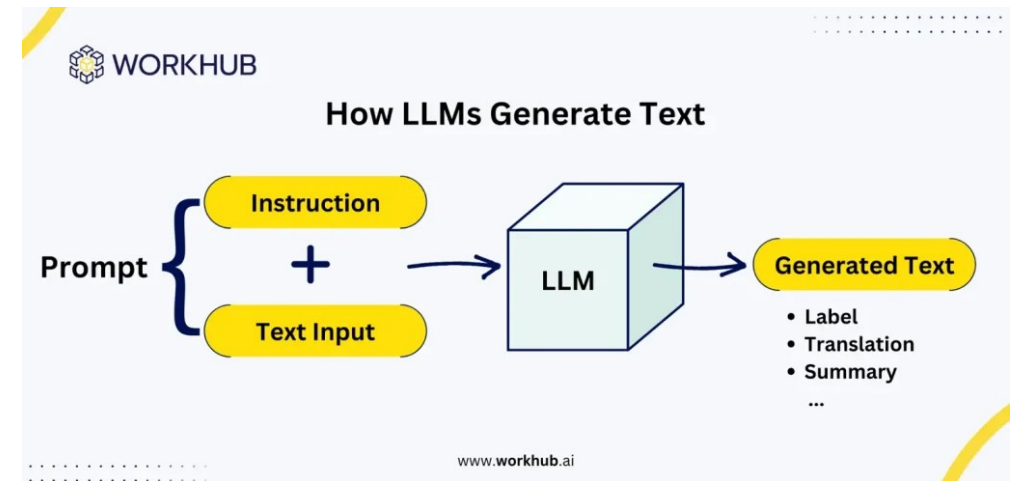
- Combining prompting and sampling
 - Structured extraction pipeline
 - System prompt defines strict JSON output.
 - User provides text.
 - Temperature = 0.1
 - Top-p = 0.9
 - Max tokens = bounded
 - Stop sequence = end of JSON
 - Produces near-deterministic machine-readable results
 - For Claude specifically
 - Consider enabling JSON mode or tool use to enforce structured output at the API level rather than relying solely on prompt instructions



Return output as a Markdown table with columns:
Feature | Description | Risk Level

Code Generation

- Task clarity
 - Ambiguous prompts produce
 - Overengineered solutions
 - Incorrect assumptions
 - Missing edge cases
 - Unnecessary dependencies
 - Weak prompt
 - “Write a function to validate emails.”
 - Strong prompt shown to the right



Write a Python function:

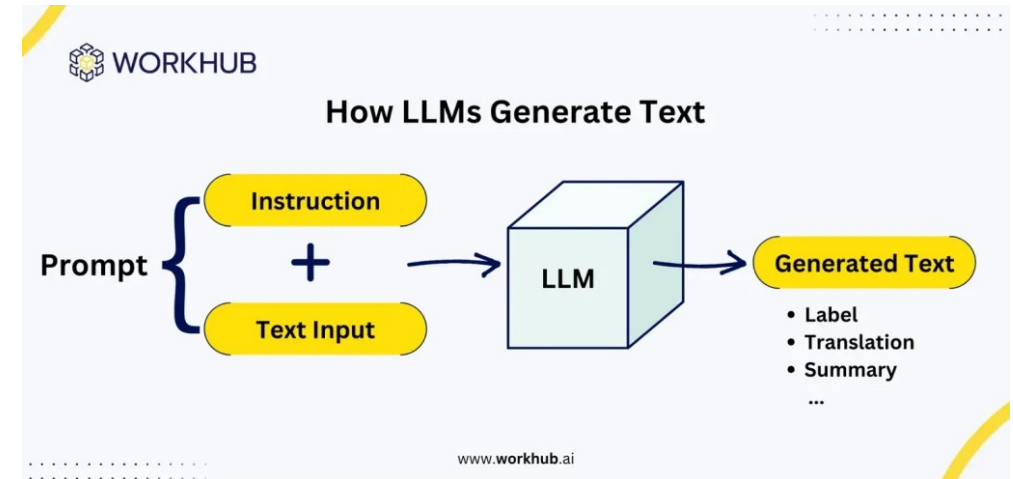
Function name: validate_email
Input: string email
Output: boolean

Requirements:

- Use regex.
- Reject emails without domain suffix.
- No external libraries.
- Python 3.11 compatible.

Code Generation

- Engineering principle
 - The more degrees of freedom you leave open, the more variability you introduce
 - Task clarity should define
 - Language
 - Version
 - Input type
 - Output type
 - Constraints
 - Environment assumptions



```
Write a Python function:  
  
Function name: validate_email  
Input: string email  
Output: boolean  
  
Requirements:  
- Use regex.  
- Reject emails without domain suffix.  
- No external libraries.  
- Python 3.11 compatible.
```

Code Generation

- Spec-driven prompting
 - LMs respond extremely well to structured specifications
 - Instead of “Build a REST API”
 - Use: the prompt shown
- Why this works.
 - The model has seen
 - API documentation
 - RFC-style specs
 - Technical documentation
 - GitHub READMEs
 - Structured prompts activate those learned patterns

Design a REST API with the following specification:

Language: Node.js
Framework: Express
Database: PostgreSQL

Endpoints:

1. POST /users
2. GET /users/{id}
3. DELETE /users/{id}

Constraints:

- Use async/await
- Input validation required
- Return JSON responses
- Proper HTTP status codes

Code Generation

- Unit test generation
 - Generating tests from spec
- The model can produce
 - Happy path tests
 - Edge case tests
 - Exception tests

Given the following function specification, generate comprehensive pytest unit tests.

Specification:

Function: `calculate_discount(price: float, percentage: float) -> float`

Requirements:

- percentage must be between 0 and 100
- price must be non-negative
- Raise `ValueError` for invalid input

Code Generation

- Unit test generation
 - Generating tests from code
 - Very useful for
 - Legacy systems
 - Coverage improvement
 - Regression testing
 - After test generation
 - Run tests
 - Detect failures
 - Feed failure back into model
 - This creates an iterative refinement loop

Generate unit tests for the following function.
Cover edge cases and failure modes.

<code block>

Code Generation

- Refactoring prompts
 - Refactoring is not rewriting
 - It is changing the structure of the code without changing its functionality
 - Weak prompt
 - Improve this code
 - Produces vague results
 - Better prompt shown on the right

Refactor the following Python function:

Goals:

- Improve readability
- Follow PEP8
- Reduce cyclomatic complexity
- Preserve behavior exactly
- Add type hints

Do not change functionality.

Code Generation

- Advanced refactoring
 - You can request
 - Convert to functional style
 - Extract helper methods
 - Apply design patterns
 - Optimize performance
 - Remove duplication
 - Structured goal-driven refactoring works best

Refactor the following Python function:

Goals:

- Improve readability
- Follow PEP8
- Reduce cyclomatic complexity
- Preserve behavior exactly
- Add type hints

Do not change functionality.

Code Generation

- Basic debugging
 - Debugging prompts require structured reasoning
 - Providing
 - Expected behavior
 - Actual behavior
 - Reproducible example
 - Significantly improves results

Follow this debugging process:

1. Identify potential error sources
2. Explain root cause
3. Provide corrected implementation
4. Suggest test case to prevent regression

Code Generation

- Secure coding prompts
 - LLMs default to functionality over security unless instructed otherwise
 - Common security prompt additions
 - “Follow OWASP guidelines”
 - “Prevent XSS”
 - “Use constant-time comparison”
 - “Avoid hardcoded secrets”
 - “Use prepared statements”
- Security must be explicitly requested

```
Write a login handler in Flask.
```

```
Security requirements:
```

- Use parameterized SQL queries
- Hash passwords with bcrypt
- Prevent SQL injection
- Validate input
- Do not expose stack traces
- Follow OWASP best practices

Code Generation

- Secure coding prompts
 - Recent Claude models tend to apply common safety practices by default
 - Parameterized queries
 - Input validation
 - No hard-coded secrets
 - For safety-critical code, do not rely on defaults
 - State security requirements explicitly
 - For high-assurance code, use prompting with:
 - Static analysis tools
 - Security-focused code review
 - Explicit threat modeling in the prompt

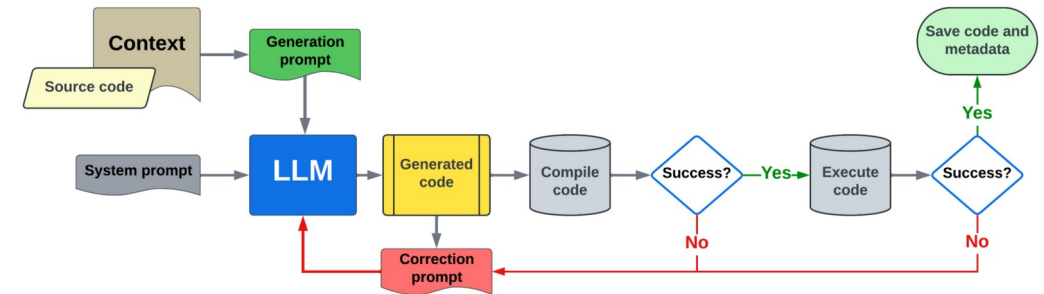
```
Write a login handler in Flask.
```

```
Security requirements:
```

- Use parameterized SQL queries
- Hash passwords with bcrypt
- Prevent SQL injection
- Validate input
- Do not expose stack traces
- Follow OWASP best practices

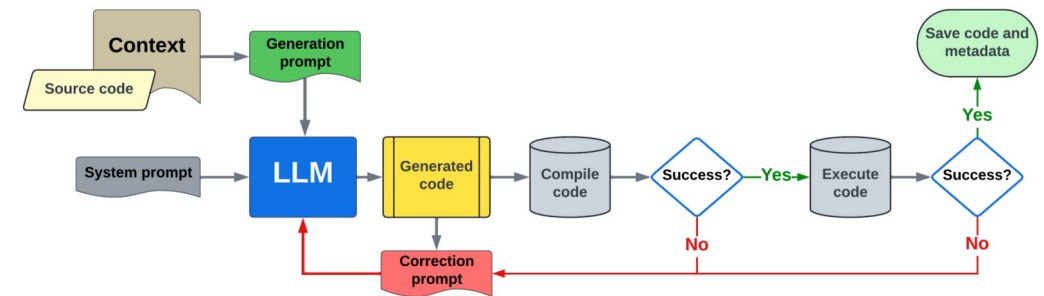
Code Generation

- Best practice AI assisted pipeline
 - Stages
 - Spec-driven code generation
 - Unit test generation
 - Run tests
 - Debug failures
 - Refactor for readability
 - Add security hardening
- Leverages LLM coding assistant at each step



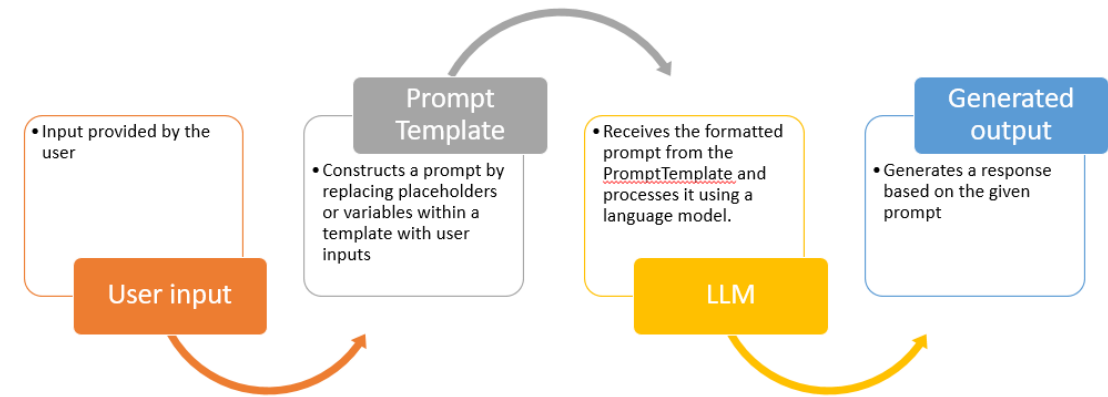
Code Generation

- This pipeline can be:
 - Implemented manually using one prompt per step
 - Implemented iteratively in a single chat
 - Automated using an agentic coding tool
 - Claude Code is Anthropic's agentic coding CLI tool that implements this pipeline directly
 - Spec → generate → test → debug → refactor, all from the terminal
 - Integrates with the user's local file system and shell
 - Available as a standalone tool and via IDE plugins (VS Code, JetBrains)



Prompt Templates

- In a production system
 - Prompts are development artifacts
 - Treated like code artifacts or test artifacts
- They should be
 - Modular
 - Reusable
 - Versioned
 - Testable



Prompt Templates

- Without templates
 - Prompts become inconsistent
 - Behavior drifts
 - Debugging becomes difficult
 - Updates cause regressions
 - Security risks increase
- With templates
 - Outputs are predictable
 - Prompts are reusable
 - Versioning becomes possible
 - Testing is easier
 - Scaling is manageable
 - Consistent with software engineering

```
system = """You are a {role}.
```

```
Follow these constraints:
```

```
{constraints}
```

```
"""
```

```
user = """<task>
```

```
{task_description}
```

```
</task>
```

```
<input>
```

```
{input_data}
```

```
</input>
```

```
<output_format>
```

```
{output_format}
```

```
</output_format>
```

```
"""
```


Prompt Templates

- Sample classification template

```
System: You are a classification assistant.
```

```
User:
```

```
<task>
```

```
Classify the following text into one of these categories
```

```
- Billing
```

```
- Technical Support
```

```
- Sales
```

```
- Other
```

```
</task>
```

```
<text>
```

```
{{customer_message}}
```

```
</text>
```

```
<output>
```

```
Return only the category name. No explanation.
```

```
</output>
```

Prompt Templates

- Summarization template

```
System: You are an executive summarization assistant.
```

```
User:
```

```
<task>
```

```
Summarize the following document in {{num_sentences}} sentences.
```

```
</task>
```

```
<document>
```

```
{{document_text}}
```

```
</document>
```

```
<constraints>
```

- Focus on key business insights
- Avoid minor details
- Do not add information

```
</constraints>
```

```
<output>
```

```
Plain text summary only.
```

```
</output>
```

Prompt Templates

- Code generation template

```
System: You are a senior Python engineer.
```

```
User:
```

```
<task>
```

```
Write a function that satisfies the following specification.
```

```
</task>
```

```
<specification>
```

```
Function Name: {{function_name}}
```

```
Input: {{input_description}}
```

```
Output: {{output_description}}
```

```
</specification>
```

```
<constraints>
```

```
- Python 3.11
```

```
- No external libraries
```

```
- Include type hints
```

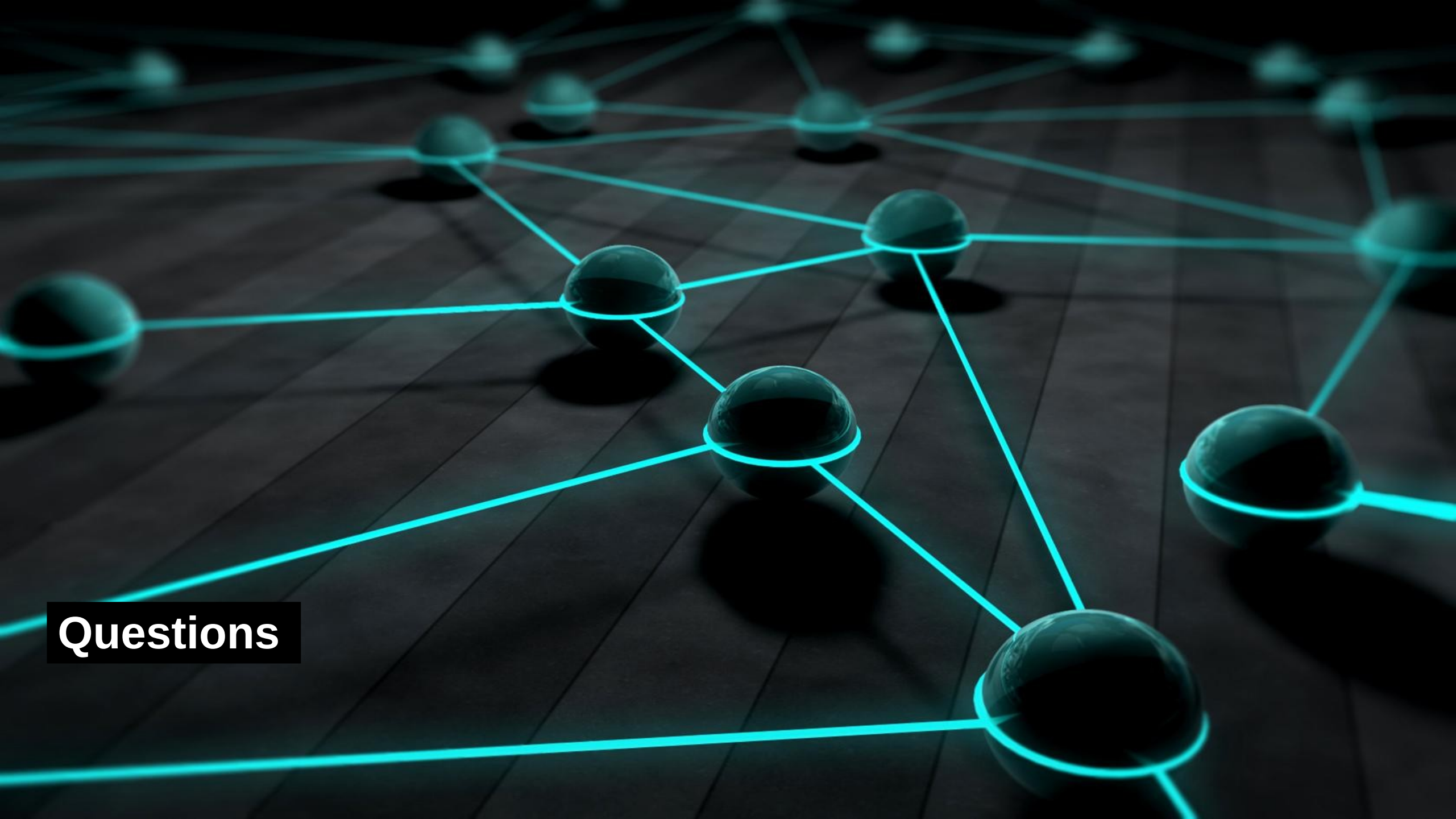
```
- Handle edge cases
```

```
</constraints>
```

```
<output>
```

```
Return only the code. No prose.
```

```
</output>
```



Questions